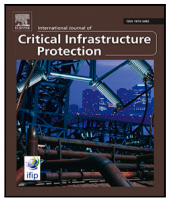
international journal of critical infrastructure protection 38 (2022) 100552

Contents lists available at ScienceDirect

International Journal of Critical Infrastructure Protection

journal homepage: www.elsevier.com/locate/ijcip



A taxonomy of IoT firmware security and principal firmware analysis techniques

Ibrahim Nadir [a,*], Haroon Mahmood [a], Ghalib Asadullah [b]

[a] National University of Computer & Emerging Sciences, FAST - Lahore, Pakistan
[b] Al-Khawarizmi Institute of Computer Science, University of Engineering & Technology - Lahore, Pakistan

ARTICLE INFO

Keywords:
IoT
IoT security
IoT firmware
Firmware analysis
Firmware security
Firmware reverse-engineering

ABSTRACT

Internet of Things (IoT) has come a long way since its inception. However, the standardization process in IoT systems for a secure IoT solution is still in its early days. Numerous quality review articles have been contributed by researchers on existing frameworks, architectures, as well as the threats to IoT on different layers. However, most of the existing work neglects the security aspects of firmware in the IoT ecosystem. As such, there is a lack of comprehensive survey on IoT firmware security that highlights critical reasons for firmware insecurity in IoT, lists vulnerabilities, and perform an in-depth review of the principal analysis techniques. This article aims to fill that gap by delivering, to the best of our knowledge, the first comprehensive review article of the firmware (in)security of IoT devices. Starting by highlighting the importance of firmware security, this research work recognizes critical reasons behind the insecurity of firmware by discussing technical, commercial, standardization, and researching aspects. In particular, the scope, evolution, and internals of IoT firmware along with their security implications are discussed. Furthermore, a taxonomic classification of IoT firmware vulnerabilities has been presented. We also discuss complications that hinder the detection of firmware vulnerabilities before doing a detailed analysis of existing vulnerability assessment tools and techniques. A comparative analysis of the principal analysis techniques is provided in terms of the vulnerabilities they discover, the methodology they employ, and the platform and/or architectures they support. Towards the end, some key research issues have been identified to encourage and facilitate research in the firmware security domain of IoT. Finally, some recommendations have been provided for the IoT device vendors, developers, and integrators.

1. Introduction

The mass production of Internet of things (IoT) devices is predicted to grow at an explosive rate [1–3]. The deployment of 5G will further hike the existing usage of IoT devices. Consequently, a great deal of investment in IoT domain is evident [4,5]. IoT is already collecting and providing huge amount of data on humans due to which the threat perimeter to humans privacy is bound to expand [4,6,7]. Furthermore, the versatility in IoT devices is such that different organizations and researchers are defining IoT in different ways[1,2] [2,8,9]. Hence, the security management is a daunting task in the IoT paradigm. Therefore, increasing dependence of humans on technology [10] does not guarantee any privacy or security. Apart from that, IoT is the fusion of multiple technologies and therefore all existing security issues have converged to IoT [11,12].

Numerous attacks have been recorded where IoT is being used as a tool to target existing infrastructure [13]. Researchers have been working on the security of IoT devices for over a decade now. However, most of the work is focused on the networking and system security side of the IoT[3] [2,8,13–19] whereas the firmware portion is often sidelined.

* Corresponding author.
E-mail addresses: ibrahim.nadir@nu.edu.pk (I. Nadir), haroon.mahmood@nu.edu.pk (H. Mahmood), ghalib@kics.edu.pk (G. Asadullah).

[1] Overview of the Internet of things https://www.itu.int/rec/T-REC-Y.2060-201206-I.

[2] C. Zavazava, ITU work on Internet of Things — ICTP workshop (2015, http://wireless.ictp.it/school_2015/presentations/secondweek/ITU-WORK-ON-IOT.pdf.

[3] Mirai "internet of things" malware from Krebs DDoS attack goes open source, https://nakedsecurity.sophos.com/2016/10/05/mirai-internet-of-things-malware-from-krebs-ddos-attack-goes-open-source/.

[4] LOJAX First UEFI rootkit found in the wild, courtesy of the Sednit group, https://www.welivesecurity.com/wp-content/uploads/2018/09/ESET-LoJax.pdf.

[5] Getting a handle on firmware security, https://uefi.org/sites/default/files/resources/Getting%20a%20Handle%20on%20Firmware%20Security%2011.11.17%20Final.pdf.

[6] Coreboot firmware https://coreboot.org/.

https://doi.org/10.1016/j.ijcip.2022.100552
Received 8 July 2021; Received in revised form 15 May 2022; Accepted 19 July 2022
Available online 25 July 2022

Firmware is a type of software in any device which interacts with the hardware. At firmware-level, the privilege on any system is the highest. In effect, if the firmware is subverted, the entire system is subverted. One white paper[4] suggests that well-known attackers (*Fancy bear*) have now started using firmware to launch deadly attacks because the security on this part of the computer stack is often overlooked.[5] Luckily, on general purpose computers there are standardized specifications (For example, Unified Extensible Firmware Interface (UEFI), Coreboot[6]) for developing firmware stack. Furthermore, according to UEFI, as of 2018 around 80–90 percent of the personal computers and servers used UEFI based firmware worldwide . Therefore, managing a single standard for virtually all the systems of a particular domain is not much of an issue. For this reason vulnerability exploitation on firmware of traditional computers have not paid much off.

Security of IoT firmware, however, is completely different. Either low-level or high-level, firmware is a significant portion of the software present in an IoT device. Therefore, the successful exploitation of any portion of firmware usually results in the complete hijacking of the device [6,13,20]. Contrary to firmware security on a general purpose computer, firmware security on IoT devices can be challenging because of a number of reasons. Firstly, IoT firmware developers have little idea and awareness of the IoT device security [14,21]. Secondly, since IoT devices are mostly on and connected to the Internet, the potential exploitation of their vulnerabilities is quite plausible. Thirdly, the exceeding number of disparate architectures makes it virtually impossible to standardize solutions and formalize tools for IoT firmware vulnerability analysis. Fourthly, the resource constrained nature of IoT devices enforces trade-off between performance and security. Finally, some of the IoT devices have no update mechanisms [14,22] and usually one firmware image is burned to thousands of devices. Therefore, one firmware vulnerability can be successfully exploited over a spectrum of devices. Even so, most firmware updates are only rich-featured versions of the previous versions instead of a *bug-fix* version. Therefore, vulnerabilities maybe carried forward to future versions and hence devices. These and other reasons further escalate the lingering insecurity in IoT. As such, existing IoT security assessments solutions suffer from the inheriting drawbacks of the analysis techniques (E.g. static analysis) plus the IoT-specific limitations that hinders the firmware reverse engineering and vulnerability analysis procedures.

Numerous solutions have been developed by researchers and IoT enthusiasts to overcome the security issues in IoT. Researchers have contributed by writing comprehensive surveys highlighting security problems and their corresponding solutions. However, even if there are solutions based on firmware security, there is an imploring need for a comprehensive survey that highlights the significance of firmware in IoT security. Furthermore, the difference between various software components (like OS, bootloader, application, and firmware) of an IoT device is usually blurred by existing literature which can lead to confusion among solution developers. Finally, most surveys focus on the security of IoT system's architecture whereas the device's software security is itself a huge attack vector. These and other reasons lead to important firmware security questions mentioned in Table 1. We argue that the resolution of IoT firmware security issues is possible through the cause and effect approach. Hence, identifying the reasons of firmware insecurity and taking a step-wise approach towards subsiding each problem can lead to a secure IoT firmware. During the course of this article, we will answer the questions in Table 1 by identifying core reasons behind these issues and in the later sections find mitigation techniques for each of the problem discussed.

1.1. Contribution

This article wishes to contribute by filling the above mentioned gaps and assist new researchers in the following ways:

- We provide a complete systematic literature review of the IoT firmware security and its assessment techniques.
- This manuscript lists down compelling reasons that lead to the insecurity of firmware in IoT devices. Furthermore, a taxonomic classification of IoT firmware vulnerabilities has been provided.
- Apart from recognizing challenges in IoT firmware reverse engineering, an up to date summary, features, and analysis of prominent solutions are discussed.
- At the end, possible research opportunities and recommendations have been provided for the resolution of firmware security issues.

1.2. Paper organization

This article is divided into 7 sections. Section 2 of this paper covers the motivation behind this work along with some background and reasons of insecurity of IoT firmware. We provide a taxonomic classification of IoT firmware vulnerabilities in Section 3. Section 4 highlights some of the principal techniques for analyzing firmware for detection of the security vulnerabilities. In Section 5 we emphasize on some of the open issues that needs to be worked upon for a secure IoT firmware. We provide recommendations for vendors, IoT developers and integrators in Section 6 and finally conclude in Section 7.

2. Motivation

Fig. 1 depicts the purpose of this article. The first important reason for writing this article is the lack of survey or analysis of existing literature review in the field of IoT firmware security. Secondly, we will explain the difference in the role and scope of an IoT and traditional computer firmware. Due to the versatility of IoT devices, the role and scope of firmware also changes. We, therefore, believe that there is a need for the evaluation of firmware security in IoT devices. Thirdly, we list down some of the compelling reasons that lead to the insecurity in IoT firmware. For this purpose, we talk about how the IoT firmware evolved over the years and what events contributed to these changes. While we discuss these three aspects of IoT firmware security, we will be answering two key questions: (1) How is an IoT firmware different and what are its security implications? and (2) What is the role and scope of IoT firmware and how did it evolve overtime?

2.1. Literature review

A number of articles have been contributed in the domain of IoT security. Broadly, we can divide them into these three categories: (1) *Generic surveys on IoT security:* These articles survey IoT security but they are not focused on the firmware aspect of IoT. (2) *Specific surveys on IoT Firmware:* These articles dedicatedly survey existing literature for IoT firmware security. (3) *Existing solutions and frameworks:* These articles focus on devising a new solution or propose framework for IoT firmware analysis but in doing so, they also partially conduct survey of existing strategies. Let us discuss each one of the above categories individually.

Generic surveys on IoT security: There have been numerous survey articles on the security of IoT in general. For example, [13,23] discuss different threats and vulnerabilities in the IoT paradigm. Furthermore, [13] provide guidelines for developing a security framework for IoT vendors. They also highlight key research areas and provide recommendations. However, these studies realize vulnerabilities through a layered approach of IoT architecture, whereas the vulnerabilities associated with the IoT device and its software components are missing. Other contributions like [24] are focused on specific topics within IoT security like control-hijacking [20], CCTV cameras security [25], IoT malware [26] etc. Even though these surveys are thorough, their firmware aspect of IoT security is generally missing.

Specific surveys on IoT firmware: The strategy proposed in [24] identifies different attack surfaces in IoT devices in the form of hardware, software and protocol interfaces and review solutions pertaining to

Table 1
Vital IoT firmware security questions discussed in different sections of this article.

Sr.	IoT firmware security question	Section
1	How is an IoT firmware different and what are its security implications?	2
2	What is the role and scope of IoT firmware and how did it evolve overtime?	2
3	What are the major reasons of IoT firmware insecurity and how did we reach here?	2,4
4	What are some of most common vulnerabilities observed in IoT Firmware?	3
5	What are the primary challenges in firmware security analysis and why do they exist?	4
6	What are the key analysis techniques and which vulnerabilities and IoT architectures do they focus on?	4
7	What are some potential solutions to these problems?	5,6

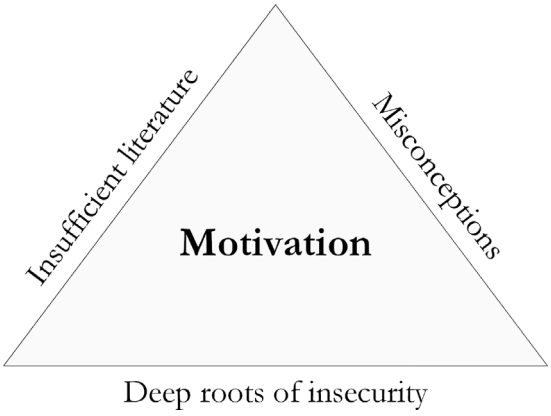


Fig. 1. The motivation triad for taxonomy of IoT firmware security.

them. However, this survey fails to identify the challenges, vulnerabilities, and reasons behind IoT firmware insecurity. A more related but brief survey is provided in [27]. The authors provide a classification of existing solutions for detecting IoT firmware vulnerabilities but lack the analysis corresponding to these solutions, the vulnerabilities covered by each solution, and the vulnerabilities associated with different types of firmware. In 2018, [28] explored the challenges that emerged during fuzzing on embedded devices. In particular, this study examined the behavior of memory-corruption vulnerabilities on various embedded devices. More recently, [29] have highlighted 28 principal challenges in emulating IoT firmware. They also reviewed numerous tools for emulation and dynamic analysis to identify vulnerabilities. Furthermore, they identified some key research areas in firmware emulation and rehosting. However, both these articles did not discuss firmware security vulnerabilities and any techniques related to static analysis.

Existing solutions and frameworks: There are still some other articles that either propose or review frameworks for IoT security [15,30,31]. Other works are pivoted on a specific solution, mostly employing one of the analysis types. These solutions employ either static analysis [6,32–36] or dynamic analysis [37–40] to conduct IoT firmware vulnerability detection. In doing so, the authors have identified some of the key challenges involved in doing static/dynamic analysis. Other than static and dynamic analysis, several fuzzing techniques have been employed to find various types of vulnerabilities in IoT device firmware [38,41]. However, such works are fixated on a particular analyses type and discuss some of the reverse engineering techniques, vulnerabilities, and challenges specific to that methodology. Furthermore, authors in [42] discuss how reverse engineering is carried out by assessing the security of 16 commodity IoT devices. Using techniques like fault injection, the authors prove that IoT devices often come with weak passwords. Apart from that, Jonas [43] discusses various tools and techniques on how to inspect IoT firmware and detect vulnerabilities in them. These and other related works [42,44,45], however, do not cover the breadth and depth of IoT firmware security.

Although things have been discussed in bits and pieces in the above-mentioned articles, a comprehensive survey is missing that focuses mainly on IoT firmware security analysis and explores various key aspects of the domain in substantial detail. Furthermore, some of the existing work is outdated as well. This article discusses and identifies the challenges, vulnerabilities, principal firmware analysis solutions, future directions, and finally provides recommendations particular to the domain of IoT firmware security. Therefore, to the best of our knowledge, we believe this article is the first to pave the way for new readers in the IoT firmware domain. Through this article, we wish to contribute by providing a broad picture of the current status of IoT firmware security and fill the existing gaps in the literature. Table 2 provides a detailed comparative analysis of the existing work with this article.

2.2. Misconception about IoT firmware

Any operation completed by a computing device requires the execution of some code. Starting from printing “hello world” on the screen to hardware communication with kernel via device drivers; a lot of such code is actually firmware. In case of IoT devices, however, the role and scope of firmware is significantly different. In certain cases, everything on an IoT device is encapsulated in the firmware including the application and the Operating System (OS) that are either burned inside the Read Only Memory (ROM) of the device or located in the flash memory of the device. Therefore, the firmware in IoT devices has a lot more to offer than that of a traditional system. Consequently, the security of firmware inevitably becomes an intimidating job. Fig. 2 provides a comparison of software stack of traditional (Fig. 2(a)) vs. different IoT devices (Figs. 2(b), 2(c)). As shown, the versatility of IoT devices has its implications on the firmware in IoT device. Resultantly, the scope and role of firmware in IoT devices vary significantly thereby making the security of IoT firmware a daunting task.

2.2.1. Scope and role of IoT firmware

Due to the ubiquitous nature of IoT devices, the internals of the IoT device can vary a lot depending upon the device resources. Thus, defining boundaries between the different software components in an IoT device is non-sensical. Because of this, three types of IoT firmware have been commonly observed [42]:

- Bare metal: These devices run with no operating system and executes burned code on the chip.
- Real-time Operating System (RTOS) based firmware: RTOS is for devices running real-time applications with limited OS features.
- Linux based firmware: Linux based firmware is used where a wide variety of functionalities are expected from a resource rich device.

Most devices use Linux based firmware as resources in IoT devices are getting beefed up [46]. The typical contents of a Linux firmware include its header data, bootloader, kernel and filesystem [12,47–49] as shown in Fig. 2(b). Fig. 2(c) shows modern day IoT devices like Raspberry pi, BeagleBone, and Arduino uno etc. which is quite similar to a traditional computer.

The architectural differences has its implications in terms of the functionalities performed by the firmware of an IoT device. Even though firmware’s role on general purpose computers have changed

Table 2
Comparative analysis of our contributions with existing literature.

Year	Ref.	Paper objective	Emphasis on IoT firmware security	Taxonomy of IoT firmware vulnerabilities	Literature analyses	Detailed analysis of existing solutions	Identification of open research issues	Recommendations
2013	[50]	Intro. to IoT firmware reverse engineering and vulnerability assessment	discussed partially as an attack vector	✘	✘	✘	✘	✘
2016	[25]	Study of attacks and mitigation techniques on CCTV and VSS	✘	✘	✘	✘	✘	✘
2017	[27]	Survey IoT firmware vulnerability detection methods and proposes a new vulnerability detection method	✔	✘	Partial discussion	✘	✘	✘
2018	[13]	Highlight the common IoT attacks with special focus on malware	✘	no focus on firmware security	✘	✘	lacks directions on IoT firmware	✘
2018	[42]	Bypassing password auth. using software and hardware techniques	Partially	✘	✘	✘	recommendations for hardware implementors only	
2018	[28]	Exploration of issues in fuzzing of embedded systems firmware with focus on memory corruption vulnerabilities	Partial	Memory corruption only	Partial	✘	✘	✘
2020	[24]	Evaluate IoT device vulnerability discovery and mitigation techniques	firmware discussed as an attack vector	✘	✔	✔	✔	✘
2021	[29]	Lists challenges in firmware emulation and analysis	✔	✘	Dynamic analysis only	✔	✔	✘
2022	This work	Highlight IoT firmware security and provide up-to-date review in terms of vulnerabilities and solutions	✔	✔	✔	✔	✔	✔

✔ = Provides coverage, ✘ = No coverage.

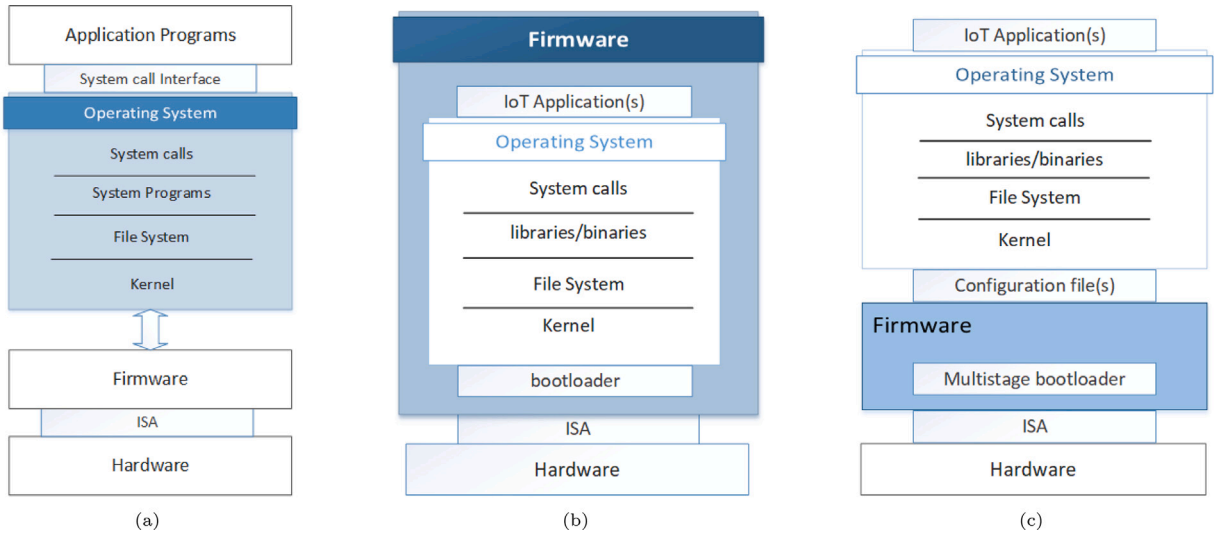


Fig. 2. Traditional vs. IoT structure (a) Traditional computer (b) Typical IoT devices (c) Resourceful IoT devices.

Table 3
Features of IoT vs. traditional firmware.

Feature/Component	Traditional firmware	IoT firmware
Fast boot-up	Desired	Essential
Prompt response	Yes	Mandatory
Reliability	Yes	Yes
Peripherals health	Yes	Not needed
Application and OS	No	Yes
Granular user control	No	Yes
Device attachments	Yes	Not needed
Network communication	Not essential	Mandatory
Middleware platform	No	Yes
Filesystem management	No	Yes
Cryptographic algorithm	Desired	Essential
Handle system updates	No	Yes
Binding with cloud/edge APIs	No	Yes
Small footprint	No	Yes

over the years but usual responsibilities include checking peripherals health, device de/attachments, boot-up preparations etc. IoT firmware, on the other hand has a lot more to offer including communication with other devices, updating itself through Over The Air (OTA) updates, implement services, contain kernel, collect logging information, maintain status of device, manage filesystems, and even encapsulate an entire OS. Depending on the type of platform used and the application of the device, the layer of a firmware in any IoT system can be quite thin in the form of an application only or it can be as thick as the entire software on the machine. Table 3 provides a brief comparison between the operations performed by an IoT device's firmware and a traditional computer's firmware. These and other similar tasks performed by the firmware in IoT devices [51] redefines security in IoT domain. Therefore, the security implications of it can neither be ignored nor tackled the same way as that of a traditional firmware.

2.3. Deep roots of insecurity

To understand the core reasons behind the insecurity of IoT firmware, we need to understand the way it came into being. Fig. 3 provides a series of events that led to the development and evolution of IoT firmware. Although the concept of IoT was given by Kevin Ashton back in 1999 [16,52], the first "Thing" on the Internet was a CocaCola vending machine in 1982 at Carnegie Mellon University in Pittsburg, USA. Way before that, Defense Advanced Research Projects Agency (DARPA) had developed WSNs which lie under the umbrella of IoT today. A number of developments occurred during the decades from 1960 to 1990 which decoupled hardware from software eventually migrating several functionalities from OS and hardware into the firmware.[7] [53] The development of RTOS in 1980s and the usage of Object Oriented Model (OOM) led to the development of reliable and robust firmware. Furthermore, modular code was being written resulting in the world seeing IoT-like-devices in the decade from 1990–2000. During the mid 1990s, Linux started getting the support which made it ideal to be used in embedded systems. Microsoft joined in the late 1990s by developing their own version of embedded Windows. The start of 21st century ensured that all the required ingredients were available to build IoT devices rapidly. These ingredients included usage of modular code like OS, bootloaders, and third-party libraries, availability of communication technologies like RFID and WiFi etc., necessary networking protocols like NAT, IPv6, and improvements in the hardware domain. Not to mention the huge amount of money being put in the IoT market by investors. Although, IoT feels like a complete success story, with each leap towards IoT, there has been a hidden consequence(s). We highlight some of those consequences in Table 4.

The issues mentioned in Table 4 are not the only issues that lead to different vulnerabilities in today's IoT firmware. The relentless development of IoT devices has given rise to numerous insecure business trends in the security ecosystem of IoT firmware. Below we highlight some unfortunate business trends that adversely affected the security of IoT firmware:

1. *Flawed business partnership:* Often times, Original Design Manufacturers (ODMs) and Original Equipment Manufacturers (OEMs) do not care about security of IoT device or firmware. Even if they do, certain code developed by ODM supplied to an OEM may have bugs or security flaw which results in the addition of a single bug to thousands of commercial products [47,55].
2. *Predetermined code base:* Manufacturers use existing old (and free) code base and libraries to reduce code footprint and time to market which usually leads to the usage of certain insecure frameworks and thus to insecure firmware [12].
3. *Backdoor installation:* At times, certain vendors themselves introduce backdoors into the firmware [12,17,32].
4. *Inexperienced developers:* Due to explosion of IoT industry, inexperienced developers are being hired by corporates to somehow

[7] Development tool evolution — hardware/firmware, https://www.eetimes.com/development-tool-evolution-hardware-firmware/.

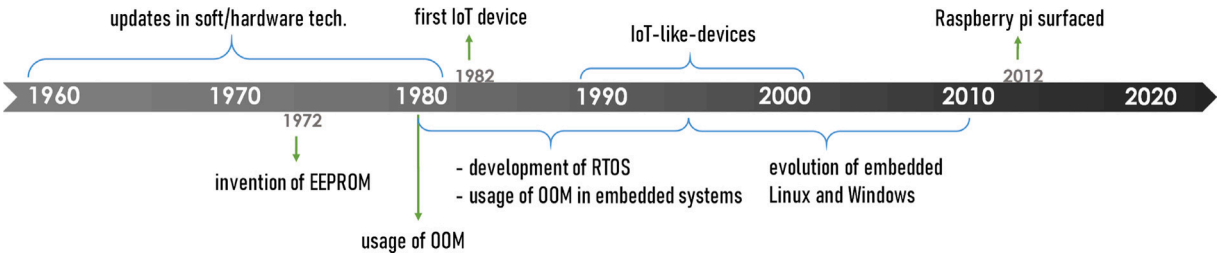


Fig. 3. Evolution of IoT firmware.

Table 4
Issues that emerged in IoT with different developments over time.

Development	Consequence
Use of OOM for reusable code	Other than elevated power consumption [54], security issues did not get fixed; rather propagated to more devices
Development of embedded Linux and Windows	Instead of developing a specialized OS for IoT, existing OSes were stripped which cannot serve the purpose
Mass production of IoT devices offering rich features	Reduced firmware development time due to short time to market. Average firmware development time increased only by two weeks between 2009–2014
Provision of extensive applications by IoT	Development of non-standardized protocols leading to insecurity
Amalgam of different technologies to build IoT devices	Inheriting security issues of all technologies in one device

reduce the time to market. This results in insecure software development [12,47].

5. *Performance vs. Security:* There is a general lack of responsibility towards security of IoT. Vendors care more about performance than security. As a matter of fact, according to a survey sponsored by IBM and Arxan 48% the IoT vendors do not test the security of their IoT applications.[8]
6. *End-user security awareness:* Consumers of the product also have little to no knowledge about security and/or privacy. Even if they do, the increasing number of IoT devices is giving them little time to prepare, adapt and learn about a product's shortcomings in terms of security [42]. On the other end, vendors often do NOT provide any security guidelines to their consumers because of their own irresponsibility towards security.
7. *Sporadic firmware updates:* Frequent updates are required to patch not only security issues but add extended functionalities. OTA may not be the desired choice of updating by vendor. In that case, if left to the user, the user may never update the firmware.
8. *Features specific updates:* Updates in firmware do not necessarily mean a bug fix or better version of firmware. Often, it is only a new feature with few lines of code which may possibly add new bugs.[9]
9. *No firmware source:* Vendors do not upload or provide their product's firmware images. They also do not provide any source code of the binary executables. For this reason, it becomes difficult for the research community to assist in finding and fixing bugs and vulnerabilities.
10. *Non-standard protocols implementation:* Certain companies implement non-standard protocols either by extending their functionality or simply modifying it to their needs thus not adhering to the specifications. This trend leads to vulnerable devices [56].
11. *Lack of regulatory agencies:* Although there are organizations that provide guidelines and best practices, there is no regulatory body that will enforce these recommendations on IoT device vendors. Even if there are regulatory agencies, they are slow in enforcing the secure practices. Vendors with little to no experience are still developing and integrating IoT devices with no embedded security in them.[10]

3. Security vulnerabilities in IoT firmware

Some of the key reasons behind the insecurity of IoT firmware have been identified in the previous section. Leveraging from that, we are now able to present the common vulnerabilities observed in IoT firmware. Some of these vulnerabilities are common to other technologies as well; which makes sense because IoT is composed of the combination of other technologies. However, this makes IoT at a higher risk of security than other technologies. We provide a taxonomic classification of IoT firmware vulnerabilities in Fig. 4. Below, we discuss each class of vulnerabilities individually:

3.1. Memory corruption vulnerabilities

Numerous consumer IoT devices have been recognized with the typical memory corruption vulnerabilities. It includes vulnerabilities like buffer-overflow, pointer violations, integer overflows and uncontrolled format strings, missing bound checks and type confusion etc. These vulnerabilities are common in IoT devices because of memory constraints thus allowing memory corruption vulnerabilities [57]. Furthermore, indispensable memory protection unit like Memory Management Unit (MMU) is also missing in lightweight IoT devices which further reduces the resistance against memory violation attacks [40]. Typically, memory corruptions cause system crashes, privilege escalation and code injection etc.

[8] 2017 Study on Mobile and Internet of Things Application Security, https://www.ponemon.org/local/upload/file/Arxan%20Report%20Final%205.pdf.

[9] Firmware Revision History |Wireless Firmware: Current Products |Firmware, https://www.lectrosonics.com/Support/Firmware-Lookup/firmware-lookup-directory.html.

[10] IoT threats: Explosion of 'smart' devices filling up homes leads to increasing risks, https://blog.f-secure.com/iot-threats/.

3.2. Hard-coded credentials

Often vendors add sensitive credentials to the firmware image without proper precautions which leads to the leakage of vital information like username passwords, certificates, data encryption keys and important URLs etc. As a matter of fact, multiple devices have been found to be using the same SSL certificates which means that if one SSL certificate is acquired, all the devices' traffic can be decrypted easily [32]. Certain devices have been known to come with hard-coded backdoors either for malicious reasons or good intentions [6,58]. It may be possible for attackers to analyze firmware of devices and use the hard-coded credentials to access the firmware or data generated by the IoT device.

3.3. Outdated (core) components

Most of IoT devices are built from outdated vulnerable nuts-and-bolts existing components [59]. These components include bootloader, kernel, system libraries along with many third-party libraries. The good thing about using these components is their availability, ease of use and in some cases reliability in terms of good coding. However, the accompanied malignancy is that these components have to be updated every now and then. For IoT devices specifically, updates are not taken seriously by vendors as well as consumer. Even if updated versions are used, new updates keep coming. In some cases, the developer gets lazy or cannot used updated components for compatibility reasons. Due to these reasons, hacking these devices becomes convenient for the attackers. During our study of various IoT firmware images, we have observed that numerous Linux Kernel vulnerabilities are carried forward in subsequent versions.

3.4. Vulnerable interfaces

A number of device interfaces are used to communicate with the IoT devices. Primarily, it includes web interface, APIs for connecting to other devices in IoT ecosystem like mobile phone, peers, servers, cloud or edge computing devices etc. These interfaces are known for their vulnerable implementation of APIs. Numerous IoT devices have been known to suffer from a number of web insecurities [11,27,37]. Since the web is a common communication interface, attackers have specially crafted malware (like IoTReaper) to exploit the web interface vulnerabilities [60]. Furthermore, for cloud interfaces, no support for two-factor authentication in IoT is a lingering problem [61].

3.5. Vulnerable network communications

Most network communications in IoT are insecure because it suffers from resourcefulness yet it has to provide diversity. The main network services and protocols include WiFi, Bluetooth, ZigBee, LoRa, 3G/4G/LTE, SSL/TLS, SSH, TelNet, TFTP etc. Some protocols are customized by vendors according to their needs which leads to insecurities. In other cases, useless ports are left open or (useless) outdated services are running on them. According to a recent report by Symantec, the highest number of attacks in IoT devices were reported on the insecure TelNet interface.[11] Although the implementation of a standardized communication model in IoT devices is highly desired for interoperability, a single vulnerability in such models can lead to a plethora of insecure devices. One such example is the Ripple20 series of vulnerabilities where 19 zero-day vulnerabilities were discovered by the JSOF research lab in the year 2020 in the Treck TCP/IP stack. This leaves millions of devices vulnerable since the Treck stack is widely used in IoT devices [62].

[11] Internet Security Threat Report (ISTR) 2019 |Symantec, https://www.symantec.com/security-center/threat-report.

3.6. Vulnerable update mechanism

A large number of IoT devices usually do not get updated by users. OTA updates are also not common in many IoT devices [63]. Even if they are updated, the mechanisms used to update the firmware is usually vulnerable. The update protocol for well-known IoT devices like FitBit have been compromised by researchers even if it is known for its end-to-end encryption [64]. Other vulnerabilities include cleartext transfer of update files, insecure update server and lack of firmware validation by end-device after receiving it [65]. Even if the research community is working on standardizing update mechanisms, it is still a daunting task to get it any time soon. Recently, Internet Engineering Task Force (IETF) has also prepared a draft architecture for secure firmware update of the IoT devices.[12]

3.7. Misconfiguration vulnerabilities

End-users are typically unaware of configuration management of IoT devices which can lead to serious issues. It may be possible to configure OS, network services, logging for information collection, notification options, web application and device default authentication settings etc. Devices are usually shipped by vendors with insecure configurations which the end-user has to modify. Even if the user is aware, it may not be possible to reconfigure every setting due to limited knowledge. Common misconfigurations observed are of iptables [26], web-servers [32], encryption, authentication credentials, BusyBox, uClibC and system bootup etc.

3.8. Authentication vulnerabilities

One of the most common vulnerability observed in IoT ecosystem is the authentication bypass vulnerability [27]. Sometimes backdoors are used by device vendors to update or manage devices remotely. Even if that is not the case, it may still be possible to bypass authentication by finding errors in authentication routines [6] to execute privileged code. Proper authentication is challenging task in IoT because of insufficient resources and required sophistication of implementation. Different techniques like fuzzing, bruteforcing and fault injection etc. are used to subvert (weak) authentication mechanism. Either IoT device vendors or end-users use common username and passwords which has resulted in millions of device hijacking cases because of malware like Mirai,[13] Silex[14] and Nyadrop.[15] Not only authentication, but authorization issues may arise if proper access control mechanism are absent on IoT devices. Such vulnerabilities usually lead to privilege escalation. In a recent report published by F-Secure, roughly 87% of the observed threats to the IoT are because of weak or default credentials 3.8. Although Mirai originated in 2016, even in 2018/19 it is the most successful attack in IoT because most of the authentication passwords are "123456" or blank passwords are used 3.8.

3.9. Non-compliance

Since IoT is still an emerging technology, new standards, rules and regulations have been introduced by different bodies like OWASP,[16]

[12] draft-ietf-suit-architecture-16 — A Firmware Update Architecture for Internet of Things, https://tools.ietf.org/html/draft-ietf-suit-architecture-16.

[13] Mirai "internet of things" malware from Krebs DDoS attack goes open source, https://nakedsecurity.sophos.com/2016/10/05/mirai-internet-of-things-malware-from-krebs-ddos-attack-goes-open-source/.

[14] New Silex malware is bricking IoT devices, has scary plans, https://www.zdnet.com/article/new-silex-malware-is-bricking-iot-devices-has-scary-plans/.

[15] IoT : Explosion of 'smart' devices filling up homes leads to increasing risks, https://blog.f-secure.com/iot-threats/.

[16] OWASP Internet of Things Project — OWASP, https://owasp.org/www-project-internet-of-things/.

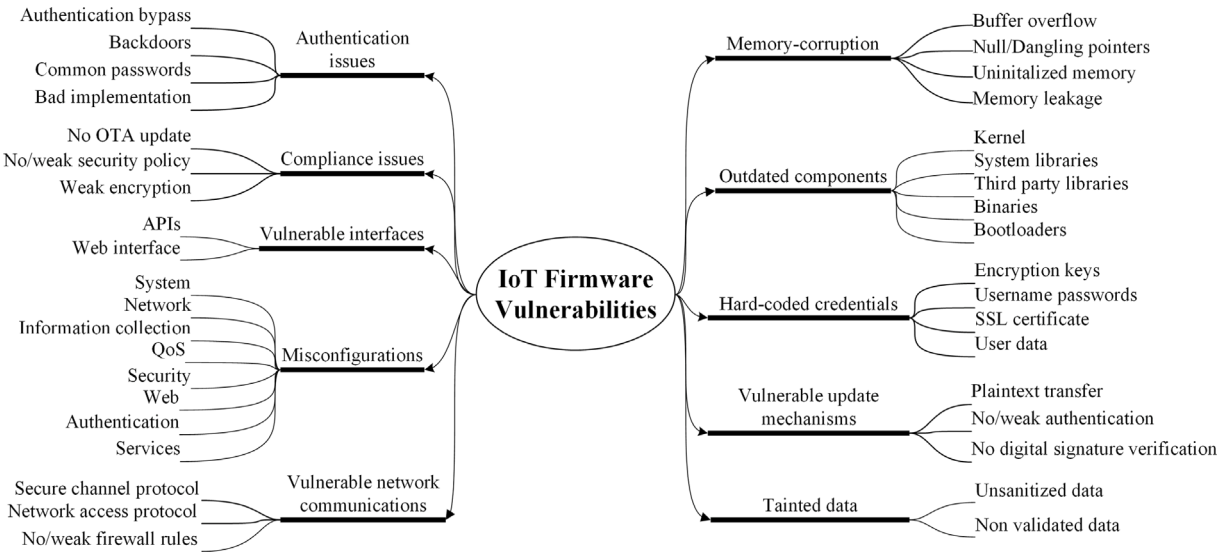


Fig. 4. Taxonomy of IoT firmware vulnerabilities.

IoT SF[17] and National Institute of Standards and Technology (NIST).[18] These bodies provide guidelines, best-practices and recommendations on how to develop and integrate a secure IoT system successfully. Typically, vendors do not follow proper guidelines and end up developing either no security policy or a vague policy which leads to poor security design. Incompetence in compliance to standards can lead to issues like data leakage, insecure booting, weak encryption algorithms, inadequate credential management and outdated services.

3.10. Tainted data

IoT systems can be interactive in nature and therefore, take, preserve, transfer, and process user's sensitive information. Proper analysis of data in the form of validation and sanitization should be carried out as this data is communicated between multiple components of an IoT system. Typical components include the IoT device, gateway, cloud/edge platforms. Taint analysis is performed to evaluate whether data in a particular form can cause a potential security breach leading to common attacks like SQL injection and cross-site scripting (XSS) [66].

4. Principal firmware analysis techniques

This section describes the most effective analysis techniques that have been developed over the years to identify vulnerabilities in IoT firmware. We refer to these solutions as *principal firmware analysis techniques*. Mainly, these techniques are based on static, dynamic, and fuzzing methodologies. We have also discussed some hybrid approaches.

In general, firmware analysis is not a trivial task as it requires the pre-requisite condition of acquiring and reverse-engineering the firmware before any analysis can be performed. Fig. 5 depicts a few steps that are performed during the reverse engineering of an IoT firmware, albeit these are not the only steps requiring effort. For instance, certain vendors customize the filesystem structure to improve security. Therefore, before diving into the analysis techniques, first, we discuss the hurdles in reverse engineering.

4.1. Reverse engineering

Reverse engineering is the process of examining the contents and behavior of a firmware [48]. Reverse engineering makes firmware analysis possible for vulnerability detection. It is a long, tedious and for the most part a manual job that requires a lot of skill [47–49,67]. The IoT firmware is usually available in a binary file format [6,51] with no source code. But before performing reverse engineering, the acquisition of the IoT device firmware is itself a challenge [6,37,67]. Acquiring the firmware is not the only issue in performing reverse engineering. Below we mention a number of challenges that act as prerequisites in firmware reverse engineering.

4.1.1. Firmware acquisition

The three primary methods of collecting firmware images are:

- *Collection from vendor:* Although most of the vendors do not provide their IoT device's firmware, the firmware of certain devices can be downloaded directly from their website [11]. Collection from websites of vendors is not humanly possible and may not be a trivial task. Even though scrappers or crawlers can be coded to download these firmwares [32,49], different vendors allow different methods of downloading their firmware images. Thus, making the task more difficult to automate [6]. For instance, certain websites generate dynamic links using Javascript to enable downloading which cannot be generated from source code of webpages. In this case, it is possible to obtain the firmware image from the FTP site of the vendor at the cost of metadata [49]. This and similar other issues make it a cumbersome task to obtain all images even if they are available on the vendors website. Therefore, there is a need for centralized repository of different firmware images [32].
- *Extraction from hardware:* Another method is to extract the firmware from the original device by debugging JTAG (named after Joint Test Action Group) or Universal Asynchronous Receiver–Transmitter (UART) port [48,58] or some other hardware techniques discussed here [42]. This, of course, comes with the prerequisite of having physical access to the device. Even if there is physical access, some IoT devices have good hardware security that does not allow dumping of the code from flash/ROM memory.
- *Grabbing during OTA update:* With the help of an OTA update, it is possible to update the entire or certain components of the

[17] IoT Security Foundation, https://www.iotsecurityfoundation.org/.
[18] Recommendations for IoT Device Manufacturers, https://csrc.nist.gov/publications/detail/nistir/8259/draft.

firmware image [15]. Many vendors do not provide a secure update mechanism for their firmware which can allow sniffing of the firmware during this update process.[19]

4.1.2. Firmware unpacking

As previously stated, the firmware is usually a binary file that encapsulates all the functionalities. The first step in reverse-engineering is the process of locating and extracting (and sometimes modifying and repacking) these functionalities from a single binary file. What makes unpacking a difficult task is the fact that there is no single way of generating this binary file. Due to the lack of standardization, different methods are employed by different vendors for packing. Some vendors have developed their own file formats for the firmware images [6,32, 38] or even a new filesystem formats [37]. Other challenges include unpackaging encrypted images [38], obfuscated images [68], monolithic firmware images where OS, kernel, IoT application and other software components mingled as a single blob [32] and images that are compressed with non-standard algorithms. Some tools exist that can make the unpacking process easy but they have their own problems. First, the output of the tools cannot always be trusted [37]. Secondly, it is a long and manual process and automating this process is not easy [6]. There are tools able to do entropy analysis of firmware images to see if there are any encryption or obfuscation techniques applied. High entropy usually means encrypted or obfuscated images [11]. Nevertheless, manual inspection is always possible using tools like HxD.

4.1.3. Decompiling

Firmware images do not come with their source code. Reading the machine code does not facilitate reverse engineering since understanding it, is impossible for humans [69]. Decompiling is the process of translating machine code to human readable high-level languages [48]. Decompiling itself is composed of a number of other steps like disassembly, lifting and data-flow analysis, control flow analysis and type analysis [70]. Disassembly is the first step as it converts the machine code into human readable low-level assembly equivalent code. It is not a trivial task and depends on a number of factors. For example, it depends on the type of compiler used. Modern compilers are able to separate executables from data in the compiled code but there exist(ed) some compilers that will confuse the decompiler by putting data in executable sections. Thereby, making it difficult to separate executing code and data. Decompilation also depends on the architecture since machine code is generated by the compiler for a specific hardware architecture. Choosing the wrong decompiler for a machine code meant for another hardware architecture will lead to failure. Traditionally, we have x86 or x64 architectures but there are an abundant number of architectures used in IoT.

The above discussed issues in disassembly phase are carried forward and later lead to errors in subsequent decompilation steps. During the lifting and data-flow analysis, assembly code is translated to higher-level internal representation. The control flow analysis determines the flow of the code using control flow graphs. The final step is to realize the data types included in the code, hence the Type analysis. Each of these steps has its own issues and pitfalls in sub steps. An issue in any of these sub steps can lead to faulty or wrong decompiled code. Therefore, decompilation of larger programs is a bigger challenge since it is difficult to decompile the entire program correctly. For this purpose, it is desirable to recognize sections of code that is of special interest. This is where debuggers can play a suitable role. Debuggers helps understand what goes on inside a program while it is being executing[20] and therefore helps recognize sections of program that are important. Thus, reducing the overall code size to inspect.

In general, decompilation is not trivial and not always successful as it has a number of aspects to it. There are however some tools that does decent level of decompilation and we have mentioned few of the tools in Table 5 such as Radare2, Interactive Disassembler (IDA) and Ghidra. Not all tools are free though. For example, IDA pro costs a lot.

4.1.4. Other challenges

The issues discussed above are not the only ones to solve during the firmware reverse engineering process. For instance, the usage of packers is one way of making the reverse engineering a tedious job. Using packers, the functionality of code remains the same but the code is systematically presented in such a way that it seems nonsensical. Packers can obfuscate or encrypt the firmware image. Although packers have in essence no effect on dynamic analysis, it makes static analysis more cumbersome [68]. Generally, malware makes use of packers to hide their identity. However, IoT vendors utilize tool to mitigate firmware reverse engineering.

Another major problem is the non-availability of metadata related to firmware images. Acquiring only the binary file is not enough. It is important to know what to expect inside a firmware image. Metadata usually contains change-log, vendor details, product name, release date and version number [49] . For example, in order to make the decompiling process easier, it is necessary to know what hardware architecture this image was compiled for. Also, if we are able to recognize the IoT device's application, we can intuitively assume a number of things about the device. For example, its network protocols, OS and libraries etc. which can provide a good insight on the overall security of the device. Some vendors also obfuscate or disable debugging ports to avoid hardware-based hacking [38,42] of the device. Therefore, it is another hurdle in the process of reverse engineering.

Fig. 6 depicts the firmware insecurity in a nutshell by compiling all noteable reasons. Mainly, classification of the prominent reasons are grouped into four categories: (1) Firmware development lifecycle (2) Reverse engineering (3) IoT generic issues (4) Other issues

4.2. Analysis techniques

There are primarily two ways of finding vulnerabilities in a system. One way is to input maliciously intended inputs patterns to the system to trigger an unexpected event. Thus, leading to breaking the system's security. This method is generally referred to as Fuzzing/Fuzz testing. Another method is using static and/or dynamic analysis. Whatever the approach is, finding vulnerabilities manually is not always possible and scalable strategy even if it has the best results [71].

This section of the paper discusses the existing solutions, the methodologies employed by those solutions, and the tools and/or techniques used to automate analysis for finding security vulnerabilities in the firmware of IoT devices or embedded devices in general. Our plan is to summarize the work in yearly fashion while trying to contain ourselves to the recent past thereby enabling the reader to know most recent updates in the field. We intend to review in a way that will lead the reader to choose the *right* techniques among all available solutions for comprehensive and scalable analysis of the IoT devices' firmware. Although not all the techniques discussed here are necessarily applied on IoT devices, they can potentially be used to detect vulnerabilities in IoT devices. We, therefore, discuss all firmware vulnerability tools and techniques briefly. For IoT firmware, primarily static and dynamic analysis are used. Therefore, prior to the techniques based on them, we first debate on their advantages and limitations for firmware analysis. Other hybrid techniques and fuzzing-based approaches are discussed later.

[19] ISTR ransomware & business 2016, http://www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/ISTR2016_Ransomware_and_Businesses.pdf.

[20] GNU Debugger, https://linux.die.net/man/1/gdb.

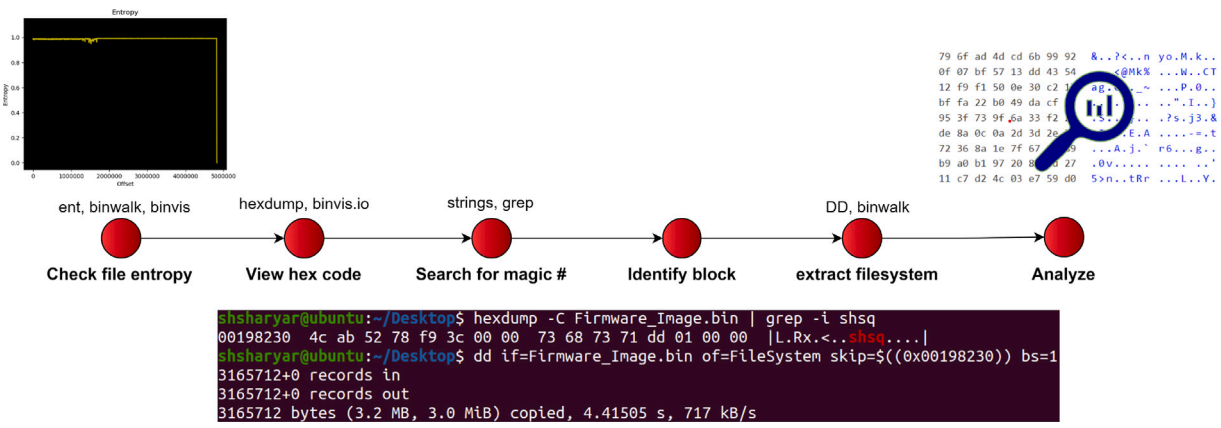


Fig. 5. Some steps of reverse engineering an IoT firmware.

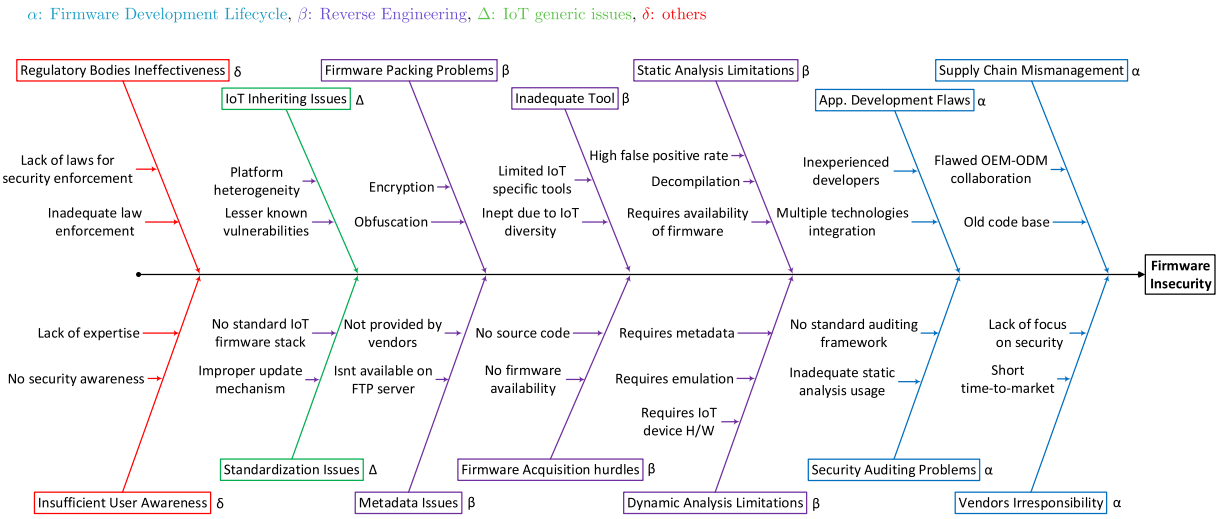


Fig. 6. Firmware insecurity in a nutshell.

4.2.1. Static analysis

Code review/analysis in any device is one of the best practices for imposing security [72]. Static analysis refers to the analysis of software without actually executing it [68]. Historically, static analysis was done to find bugs in code in the form of a utility named lint [73]. With time, static analysis extended its utility and now it can be used for a number of reasons including security analysis. Static analysis has the advantage that it does not require physical access to the device whose firmware needs to be analyzed [32]. Hence, from this perspective it is a scalable solution. Typical vulnerabilities found using static analysis include invalid references, buffer overflows, memory corruptions flaws [74], segmentation faults and uninitialized variables [32].

Before accepting the challenge of static analysis, the acquisition of source code or at least decompiling of the code is a necessary step. Not only that, decompiled code cannot recover all compiled information [68]. Also, static analysis is a costly process if done manually and therefore from this perspective it may not be scalable after all. To solve this problem, employing tools is a good option to automate the process. Although it is easy to find errors using tools, it generates a lot of false positives [32]. Still depending upon the type of static analysis in a particular tool (For example, pattern-based static analysis,[21]) the number of false positives generated can be greatly reduced. Static analysis has the advantage that it can read through the entire code without skipping any portion. However, static analysis cannot be done if code is obfuscated or encrypted as per say [58]. The right place to embed static analysis in the IoT security ecosystem is when the source code is being developed. Thus, fully utilizing the security offered by static analysis techniques. We provide a comparison of static vs. dynamic analysis in Table 6. The first column exhibit the disciplines in which static and dynamic analysis are compared. Columns 2 and 3 describe how each analysis type is better than the other.

Static Analysis Solutions: In 2012, Grace did a static analysis of applications that came as a part of the stock android firmware image [75]. Although, the work itself was not intended to find firmware vulnerabilities, it did however find permission leaks in the pre-loaded applications using static analysis. After extracting the Dalvik bytecode of the pre-loaded application, a control-flow graph (CFG) is built to observe possible permission leaks by going through all possible paths of the code. The work was based on Android's permission-based security model. The tool was termed as woodpecker but it was not a large-scale analysis since only 13 different android phone images were inspected for pre-loaded applications in firmware.

In recent times (2014), the work done by Costin has laid the basis for firmware vulnerability detection [32]. Costin did a large-scale analysis of more than thirty-two thousand (32,000) firmware images. After having these images statically analyzed, Costin was able to detect over 693 different vulnerabilities out of which 38 zero-day vulnerabilities were discovered. The work suggests that the overall affected number of

[21] How Does Static Analysis Prevent Defects and Accelerate Delivery, https://blog.parasoft.com/how-does-static-analysis-prevent-defects-and-accelerate-delivery.

Table 5
Firmware reverse engineering & analysis tools.

Tool	Features	Speciality	Limitation(s)
Binwalk	Firmware analysis, extraction, filesystem identification, comparison, entropy analysis	Reverse engineering	cannot extract every firmware, does not support recursive unpacking, no magic byte of some filesystems in its signatures
BANG[a]	Recursively unpacking and extraction of over 130 file types, reports kernel and busybox issues.	Reverse engineering	is not matured enough to replace binwalk, inconsistent support
Radare(2)	Binary analysis, disassembling and debugging	Reverse Engineering	steep learning curve, difficult to analyze complex programs
Binary Ninja	Binary analysis with intermediate language supporting multiple platforms, offers GUI	Reverse Engineering	closed source, no dynamic analysis support
FMK[b]	Firmware unpacking and extraction and repacking, specific to Linux based routers	Reverse Engineering	insufficient support, support for linux distros only
ANGR	Framework for binary analysis using Control Flow Graphs (CFG), concolic execution	Static and Dynamic Analysis	complex to use, limited Windows binary support
FAT[c]	Built on top of five different analysis and reverse engineering tools for identifying vulnerabilities	Reverse Engineering and Dynamic Analysis	compatibility issues among its base tools and Linux
IDA Pro	Powerful interactive disassembler, debugger, support for multiple architectures	Static and Dynamic Analysis	high cost, closed source, unpolished GUI
FACT[d]	Automatic, extendable basic firmware analysis, firmware comparison	Reverse Engineering, Binary Analysis	uses static analysis only, runs on limited linux OSes, requires beefy system specification
QEMU	Efficient open source emulator, virtualizer for Linux platforms	Dynamic Analysis	contains bugs varying OSes, no GUI, good supports Linux only
Ghidra	Open source, analysis and decompiler tool, support for multiple processors, user-interactive and automated mode	Reverse Engineering	supports limited architectures and debuggers, slow performance
AFL[e]	Security oriented brute-force fuzzer employing genetic algorithm.	Dynamic Analysis	requires target input to learn and improve, no network service fuzzing or background daemons
KLEE	Symbolic VM based on LLVM[f] compiler supporting uClibc.	Dynamic Analysis	cannot run loops with symbolic conditions, consumes higher resources, lacks models for system environments
Frimadyne	Scalable, automatic dynamic analysis of filesystem for webpages, SNMP information, vulnerabilities	Dynamic Analysis	limited hardware support, cant handle emulation failures

[a]Binary Analysis Next Generation.
[b]Firmware Mod Kit.
[c]Firmware Analysis Toolkit.
[d]Firmware Analysis and Comparison Tool.
[e]American Fuzzy Lop.
[f]Low Level Virtual Machine.

devices connected to the internet (and accessible) are above 140,000. The analysis performed no new technique of static code analysis, instead a correlation engine was used to find vulnerabilities and then use that information to detect vulnerabilities in new devices that were affected by the same vulnerabilities. Even though no new technique was used, the correlation allowed scalability of the solution. Some of the vulnerabilities found were access to RSA keys, authorized SSH key files to remotely connect to other terminals, hash values of passwords burnt onto firmware etc.

Firmalice [6] introduced in early 2015 by Shoshitaishvili, is another binary analysis tool that checks an embedded device's firmware for vulnerabilities. It is able to slice a program and use symbolic execution engine to execute parallel functions for authentication bypass vulnerabilities. A good feature of Firmalice is that it is scalable. However, by scalability we mean parallel firmware slice computations to arrive at a privileged point in code unlike the work done by Costin where automatic unpacking and vulnerability detection of many firmware images was carried out simultaneously. Firmalice can be considered rather intelligent static analysis technique because in order to understand that an instruction is privileged, it needs to understand the "Security Policy" of the device. Another good feature of Firmalice is its architecture-independent analysis of the firmware supporting a number of architectures including both ARM and PPC [76]. (In 2016) Shoshitaishvili, along with his team created a tool called ANGR [10] that is effective both for static and dynamic analysis. ANGR is used by a number of other tools to carry out firmware analysis. Later in 2015 another open source tool called PIE(Parser Identification in Embedded Systems) [9] was introduced to detect functions with parsers components and complex code. Before any classification can be done of the parsers components, the binary code of the firmware is converted to intermediate language via LLVM. PIE is able to analyze the embedded systems' firmware without any documentation or source code. This is a significant achievement because the versatile nature of embedded or IoT devices makes it very difficult to collect the metadata and understand the nature of the content under analysis. PIE can be used for detecting exploitable bugs, extract protocol specifications and finding hidden commands. PIE can be considered as a good potential candidate for IoT devices' firmware auditing because it was tested on four widely used devices: GPS, Power meter, Hard disk (HDD), and PLC.

Using CFGs, firmware vulnerabilities can be effectively analyzed but the solution is not scalable for firmware analysis. In 2016, Feng and his team came with an effective algorithm called `Genius` [77] for solving the scalability issue with Control Flow Graphs (CFGs) using the combination of Machine Learning and Computer Vision. To top that, the work done by Xu in 2017 uses a neural network-based approach to outperform `Genius` [78]. The authors developed and tested their prototype called `Gemini` to reduce its training time as well as find significantly more vulnerabilities in firmware images. In 2019, Wang suggested a two-stages firmware vulnerability detection based on code similarity [79]. Wang compares their work with the `Gemini` framework and argues that their solution is both accurate and efficient.

A common issue with firmware development is the undocumented credentials or (purposefully left) backdoors that come with it and there

Table 6
Comparative analysis of static and dynamic firmware analysis techniques.

Discipline	Static	Dynamic
Methodology	Analysis performed by statically analyzing code	Behavior of program is observed by executing it
IoT Device	Not required for static analysis	IoT device is required for dynamic analysis
Scalability	Scalable solutions can be developed if large scale of firmware repository is available	Scalable solutions cannot be achieved without virtualization
Metadata	Desired but not necessary	Necessary
Virtualization	Not required	Emulation may fail even if firmware and metadata is available due to multiple reasons (see section 5.5)
Decompilation	1. Not possible due to unavalability of sepecific decompilers 2. Decompilation is not accurate	No need for code decompilation in dynamic analysis
Runtime insights	Does not provide insight about the code/data or complex problems	Can provide insights about code/data while executing
Unused code	Possible to analyze unused code	Not feasible to analyze unused code
False positives	Higher rate of false positives	No issues of false positives
Vulnerability detection	Can identify vulnerabilities like memory-corruption, segmentation faults, uninitialized variables etc.	Can detect any vulnerability when code is executed
Firmware acquisition	Firmware acquisition is necessary for static analysis	No need to acquire firmware if actual device is (remotely) available
Runtime issues	Static analysis cannot capture runtime vulnerabilities	Dynamic analysis can capture runtime vulnerabilities
Encryption/Obfuscation	Hinders static analysis	Not an issue for dynamic analysis
Unexploitable code	Static analysis can find vulnerabilities that are unexploitable	Cannot find unexploitable code

has advantage over the other.

is usually no way to know it. Stringer [59], a tool based on automatic static analysis of firmware addresses this issue. Using Stringer, Thomas and his team were able to analyze 7590 different firmware images. The team was able to find three backdoors, two of which were new along with the identification of static data processing routines. Apart from it being lightweight, another good feature of this work is its application on a large-scale thereby reducing the manual human effort for static analysis.

Thomas, again in 2017, developed another system called HumID-Ify [35], which searches for undocumented functionality hidden in the firmware. Additionally, it is able to find authentication bypass vulnerabilities. The unique feature about this research work is the usage of machine learning to identify hidden functionality. The authors create a set of profiles with expected firmware behavior using a Binary Functionality Description Language (BFDL) which is later used to compare with the real-time behavior of the code. If any serious changes are observed, the firmware is supposed to have hidden functionality. A set of 800 different firmware images were used to train the semi-supervised classifier and then 100 test cases where taken which results in an efficiency of 96% with almost no false positives. Although it is a novel approach, it cannot be regarded as a complete approach because it requires expert human knowledge and metadata of the firmware otherwise a lot of false positives will be generated.

Recently in 2018, an attempt was made on newer firmware images by FirmUp [36] to find vulnerabilities using static analysis. The authors found 347 vulnerabilities in 147 new versions of firmware using CFGs and slicing technique to find exact location of vulnerable procedures. Using tools like Binwalk, IDA Pro and ANGR, the authors claim to have outperformed the state-of-the-art detection rate by a margin of 45% on average.

Most recently in 2018, Chin-Wei Tien came up with the Universal Firmware vulnerability Observer (UFO) [80] which is a firmware vulnerability discovery system. As such, it has a reputation for reversing firmware, leaking passwords and finding backdoors using a newly developed algorithm called Shell Script Dependency (ShDep). UFO makes sure that the embedded IoT device is following IoT specific security and privacy standards like OWASP, UL-2900[22] and ICSA Labs.[23] According to the authors, 96% of the 237 devices were reverse engineered which concluded that more than 70% of the devices had common vulnerabilities. One limitation of the UFO is that it is not able to reverse engineer firmware for filesystems if the firmware is obfuscated or encrypted. Yet, UFO claims to have better firmware filesystem extraction capability as compared to Fimrware-Mod-Kit.

A comparison of the principal analysis techniques based on static analysis has been provided in Table 7.

4.2.2. *Dynamic analysis*

In the previous section we discussed a number of positives of the static analysis. Although it has its advantages, one the biggest flaw is that it is not possible to test and exploit the system without having the device itself. For this purpose, either the device has to be present for testing or we can emulate the firmware using different tools like KLEE [81] or QEMU [82]. Contrary to static analysis where firmware is dissected and code is observed/read, it may be possible to execute the code and observe its behavior. In such cases, it is not a necessity to know the internals of a program. Only random inputs maybe provided to know the behavior by observing the output generated by the program. However, dynamic analysis of firmware images requires metadata of a firmware to be emulated so that the required environment may be setup. Because one cannot always emulate a firmware image without any knowledge of the underlying architecture used to build that firmware. Dynamic analysis may be used when no source code is available or decompilation is not successful. For example, for

[22] Standard for Software Cybersecurity for Network Connectable Products, https://standardscatalog.ul.com/standards/en/standard_2900-1_1.

[23] IoT Security & Privacy |ICSA Labs, https://www.icsalabs.com/technology-program/iot-testing.

Table 7
Assessment of static analysis based solutions.

Reference/Tool	Analysis method	Target vulnerability	Scalability	Versatility	Mines new vulnerability	Supported architectures
Costin[32]	Correlation Engine	any	✔	✔	✔	multiple
Firmalice	Symbolic execution	authentication bypass	✔	✔	✘	multiple
Genius	Control flow graphs	any	✔	✔	✘	multiple
PIE	Parses identification	parses vulnerabilities	✘	✔	✘	multiple
HumIDIfy	Machine learning	finding backdoors	✘	✘	✔	multiple
FirmUp	Program slicing	multiple	✔	✔	✘	multiple

Xtensa architecture, there is no real known decompiler. In that case, dynamic analysis is the only option.

Dynamic analysis has its limitations too. It does not scale well in the case of firmware analysis because automatic emulation of the firmware is not always possible as emulation of different devices requires customized configuration setups.

Dynamic Analysis Solutions: FIE [83] is an open source tool introduced in 2013 that is built on top of symbolic execution engine called KLEE [81]. FIE, however, is specific for the MSP430 microcontrollers' family. FIE has been tested on 99 different firmware images with different families of MSP430. FIE has been developed to take special care of memory locations for peripherals and invocation of interrupt handlers which helps analyze the behavior of peripherals in a flexible manner. FIE was able to find 21 bugs of all the 99 firmware images that belonged to 13 distinct MSP430 microcontrollers family. A unique technique used in FIE is to do state pruning which helps remove redundant state executions for even small firmware images. For this purpose, FIE keeps check of all the previous states analyzed. Another technique used by FIE is memory smudging in which FIE recognizes loop counters automatically to replace them with symbolic variables to help code coverage.

Avatar [58] introduced in 2014, is a unique framework that is able to do dynamic analysis of embedded devices firmware. Using Avatar, the firmware instructions are emulated while the I/O operations are channeled to the device's hardware. Hence, the firmware gets emulated on the external machine. Avatar has been applied in three different security scenarios — Reverse engineering, Vulnerability discovery and backdoor detection.

Avatar was tested on three different class of hardware devices one of which was a GSM phone, second a hard disk bootloader and third a wireless sensor node. Once the basic setup is complete, QEMU [82] — an open source emulator is used to emulate the firmware. The famous symbolic execution engine, KLEE is able to run different paths of the firmware to search for any vulnerabilities. The problem with Avatar is its usage of real hardware to discover vulnerabilities. On the other hand, it is also very slow to execute the entire procedure from the emulator to actual hardware. It is therefore, not a scalable solution. In February 2016, Daming presented the first automatic dynamic analysis system called Firmadyne [49]. The system introduced focuses on Linux-based firmware vulnerabilities in network connected devices. Firmadyne successfully analyzed a dataset of 23,035 firmware images and was able to confirm that 887 firmware images were vulnerable to one or more exploits. The system is able to automatically crawl vendor websites for firmware images along with its metadata using manually written scripts. At its heart, Firmadyne uses a customized version of Binwalk to extract the kernel within the firmware image. The system then employs QEMU to emulate a Linux kernel with customized settings according to the firmware image under analysis. Next, a number of dynamic analyses methods can be carried out to find and exploit vulnerabilities on the emulated firmware. Firmadyne also uses the popular Metasploit framework [84] to exploit vulnerabilities. Two improvements in the Firmadyne system can be (1) the inclusion of analysis of non-Linux-bases systems (E.g. RTOS) and (2) improvements of firmware extraction schemes to include obfuscated and encrypted firmware images.

After his work on static analysis of firmware, Costin implemented a fully automated dynamic firmware analysis framework [37] in 2016. The solution designed by Costin uses full system emulation (via QEMU) without the usage of actual hardware. The analysis aimed at finding vulnerabilities in Linux-based systems by testing the embedded web interfaces by using tools like Arachni, Zed Attack Proxy(ZAP) [85] and w3af [27]. The work allowed flexibility for utilizing even more common tools like Nmap [86] and Nessus [87] for scanning and Metasploit for exploiting vulnerabilities. Although, the dynamic analysis carried out resulted in discovery of 225 new vulnerabilities in 45 different firmware images, the authors claim that a lot improvement is possible by enhancing the analysis tools used and also perfecting the emulation of firmware since dynamic analysis has its inheriting automation issues.

In 2017, Palavicini worked in the Industrial Internet of things devices to find firmware vulnerabilities [88]. Their near automated work focuses on finding vulnerabilities in industrial control systems employing a number of tools to do the analysis. The comprehensive work makes use of binary analysis tools such as ANGR, the cyber reasoning system (CRS) 'Mechanical Phish'[89], American Fuzzy Lop (AFL) [90], and virtualization tools such as OpenPLC [37], Firmadyne, and QEMU to discover vulnerabilities. The work is sliced in three steps which starts with extraction of the firmware blob to extract code for emulation. In the second phase OpenPLC [91], QEMU and Firmadyne are used to emulate the code after extraction. Finally, the code is analyzed for vulnerabilities using a number of techniques like fuzzing and symbolic execution. This analysis results in finding backdoors, information leakage and code for creating botnets. Recently, in 2019 a new framework called the `FIoT` was proposed to find memory corruption issues in constrained IoT devices firmware. `FIoT` uses symbolic execution to fuzz test firmware slices for memory corruption issues on a variety of processor architectures. `FIoT` however slices only special parts of code which can be linked to buffer overflows vulnerabilities. Furthermore, symbolic execution is a costly operation and therefore this solution cannot be applied to beefy firmware images. The authors were able to discover 35 zero-day memory corruption vulnerabilities in 115 lightweight IoT devices.

We have provided a comparison between the principal analysis techniques employing dynamic analysis in Table 8.

4.2.3. Hybrid analysis

This section will include systems that uses both static and dynamic analysis techniques to discover vulnerabilities in IoT or embedded systems. In a practical sense, multiple systems are actually hybrid — For instance, Costin's work [37] in the previous section is actually a combination of dynamic and static analysis to achieve full automation. Still, we discuss some systems briefly that we believe are purely hybrid by nature.

DroidRay [92], is a system developed in 2014 to discover malicious code and applications in the firmware of Android devices. DroidRay has

Table 8
Assessment of dynamic analysis based solutions.

Reference/Tool	Analysis method	Target vulnerability	Scalability	Versatility	Mines new vulnerability	Supported architectures
Firmadyne [49]	Emulation	multiple	✔	✔	✔	ARM, MIPS
FIE	Symbolic execution	memory safety bugs	✘	✘	✘	MSP430
Avatar	Emulation	any	✘	✔	✔	multiple
Costin[37]	Emulation	web vulnerabilities	✔	✔	✔	multiple
FIoT	Symbolic execution	memory corruption	✔	✘	✔	multiple

been applied on more than 250 Android firmware images to discover that around 8% of the pre-installed applications have vulnerable signatures and 7.6% firmware images have malware traces. The authors have both statically and dynamically collected the APK of different applications. Statically, the APK of applications are extracted from the firmware image by checking for AndroidManifest.xml file while dynamically, the firmware is flashed on the android device and execute a number of commands to extract the APK files from the firmware. Furthermore, the signatures of these applications are analyzed and scanned for viruses using static analysis. DroidRay also searches the iptables commands statically and executes the iptable-list-rules command dynamically to search for malicious traffic rules.

Another hybrid analysis is carried out on the security of Fitbit tracker firmware in 2018. Even though the Fitbit system including the cloud, app and tracker uses high-end encryption, there are a number of vulnerabilities found in the system after a comprehensive analysis of the Fitbit ecosystem including its firmware. During static analysis, IDA pro is used to detect that the encryption scheme of XTEA is used in the EAX mode. Also, other encryption parameters were observed which led to successful decryption and even generation of encrypted messages including a message authentication code (MAC). The authors have successfully compromised the firmware update protocol dynamically to disable all security on the device including encryption and authentication. In 2019, [31] used an automatic vulnerability detection framework `AutoDES` and validated it by exploiting the identified vulnerabilities. The authors provide a comparison with `Mechanical Phish` and claim that `AutoDES` their solutions is better in terms of performance, resource scheduling, vulnerability detection and exploitation.

4.2.4. Fuzzing based techniques

In recent years, fuzzing has caught the eye of researchers for finding vulnerabilities in IoT firmware. Table 9 shows an increasing trend of fuzzing-based techniques in recent years. Let us analyze some principal techniques below.

IoTFuzzer [38] is one such framework introduced in 2018 which is based on the philosophy that the application should be fuzzed to locate vulnerabilities in IoT firmware. With a black-box approach, IoTFuzzer detects possible memory corruption vulnerabilities in the IoT devices using protocol-guided fuzzing by sending probing messages from the mobile applications of IoT devices. The prototype developed is an automatic fuzzer that when tested on 17 real-world devices found out 15 vulnerabilities including 8 new vulnerabilities. The framework works in two phases where the first phase is to understand the mobile applications of the IoT devices. In the second phase, consistent fuzzing is carried out by sending probing messages to the IoT device. The IoT device is monitored for crashes dynamically and the resulting crash messages from the device is stated.

Zheng implemented a grey-box fuzzer called Firm-AFL [93] which provides support for the POSIX library. This IoT firmware fuzzer achieves higher throughout using a technique called augmented process emulation. However, Firm-AFL is based on AFL and Firmadyne which means that any firmware that cannot be emulated through Firmadyne cannot be fuzzed with Firm-AFL. Otherwise, Firm-AFL is capable of effectively finding 1-day and 0-day vulnerabilities. Even though it supports MIPS and ARM architectures, Firm-AFL only supports POSIX based operating systems.

In 2020, FIRMCORN [41] was introduced to fuzz test the IoT firmware for vulnerabilities. The authors have developed a vulnerability-oriented fuzzing mechanism which uses a vulnerable-code search algorithm to find vulnerabilities in IoT firmware. Furthermore, the authors have proposed an optimized virtual execution to combat against the instabilities and inaccuracies during emulation process. FIRMCORN uses Unicorn Engine [94] and supports four architectures namely x86 32, x86 64, ARM 32 and MIPS 32. Apart from that limitation, FIRCORN provides support only for Linux based firmware. The authors claim that FIRMCORN is able to find two zero-day vulnerabilities in a given firmware in two hours. Nevertheless, the inheriting scalability issues with dynamic analysis will continue to haunt fuzzing based techniques.

4.3. Discussion

We provide Fig. 7 to indicate the number of solutions contributed since 2014. Most of these solutions, however, are specifically for IoT firmware analysis rather than any type of software analysis. We also show the number of solutions that use machine learning techniques to enable their solutions to evolve with the time. Furthermore, we provide Table 10 to identify solutions available against each class of vulnerability along with the platforms and the architectures they support. Careful analysis of the data has led us to the following conclusions:

- Most of the solutions are based on static analysis rather than dynamic analysis due to the tight dependence on hardware for dynamic analysis [56]. However, in the recent years, there is an increasing trend of developing solutions based on Fuzzing. Some examples are [38,41,93,98,101,107,111,114,117,126–128] etc. This shows that frameworks can be developed using existing solutions to cover many different classes of vulnerabilities.
- Most of the solutions are developed for Linux platform even though the number of Linux based devices have comparatively reduced over the years. Still, there is a need to develop solutions for bare metal and Windows based firmware.
- Even though most of the solutions support ARMS and MIPS architectures, there is support available for other architectures as well. However, different solutions supporting different architectures cover different classes of vulnerabilities and therefore, not all good solutions can prove to be effective against a vulnerability. Having said that, we cannot rely on one solution for every IoT device due to the versatile number of architectures and platforms in IoT.
- There is a lot of focus on some classes of vulnerabilities like authentication bypass, memory corruption and hard-coded credentials. But there is a lack of attention towards certain classes like misconfigurations, tainted data, compliance issues and detection of vulnerable update mechanisms. Furthermore, existing

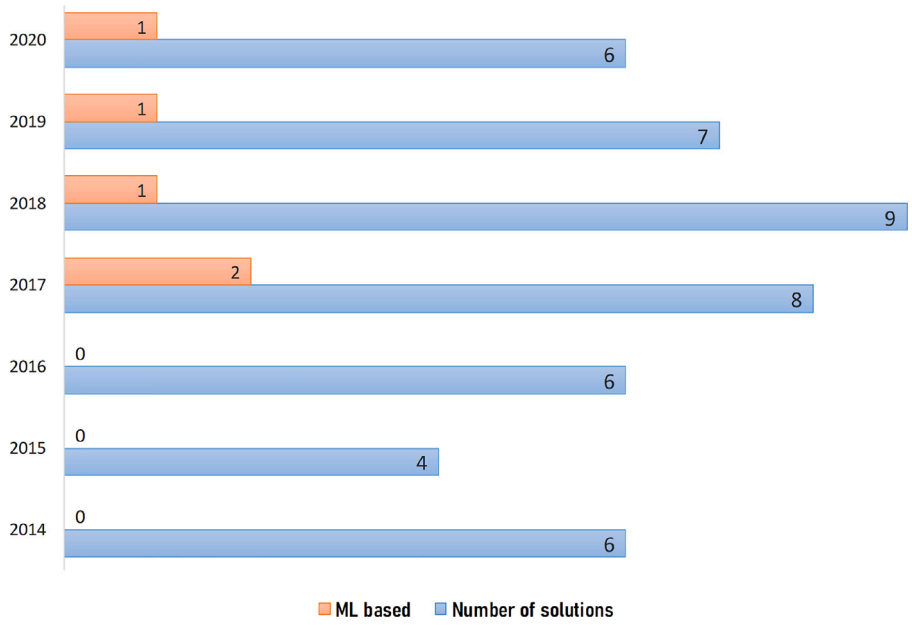


Fig. 7. Number of solutions for IoT firmware security contributed per year since 2014.

Table 9
Increasing trend of fuzzing based solutions in recent years.

Year	Number of fuzzing-based solutions	References
2016–2018	5	[95],[96],[88],[97],[38]
2019	7	[93],[98], [99],[100],[101],[40],[102]
2020	7	[41],[103],[104],[105],[106],[107],[108]
2021	10	[109],[110],[111],[112],[113],[114],[115],[116],[117],[118]

Table 10
Critical analysis of existing static and dynamic solutions in terms of vulnerability coverage, supported platforms and architecture.

Vulnerability	Static	Dynamic	Supported platforms				Supported architectures			
			Linux	Windows	RTOS	Other	ARM	MIPS	x86	Other
Authentication bypass/Backdoors	[6,32,35, 80]	[88]	[6,35] [32,80, 88]	[6]	[6]	[6]	[6,35] [32,80, 88]	[6,35] [32,80, 88]	[6,88]	[6,88]
Weak passwords	–	[32,35, 80]	–	[32]	–	[32,35]	[32,35]	[32]	[32]	
Memory corruption	[34,36, 119]	[38,40, 93]	[38,40, 119] [34,36, 93]	[38,40] [34,36]	[38,40] [34,36]	[36,38, 40] [34,93]	[38,40, 119] [36,93]	[38,119] [34,36,40, 93]	[34,36,38, 40]	[36,38, 40]
Web application	[32]	[33]	[32,33]	–	[32]	–	[32,33]	[32,33]	[32]	[32,33]
No/Weak firewall rules	[35]	–	[35]	–	–	–	[35]	[35]	–	–
Taint-style vulnerabilities	[119,120]	–	[119,120]	[120]	[120]	[120]	[119,120]	[119,120]	[120]	[120]
Hard-coded credentials/data	[32,35, 121,122]	–	[32,35, 121,122]	–	–	–	[32,35]	[32,35]	[32]	[32]
Information leakage	–	[88]	[88]	–	–	–	[88]	[88]	[88]	[88]
Insecure interfaces	[80]	[34]	[34,80]	–	–	[34]	[34,80]	[34,80]	–	-
Outdated components	[123, 124][a]	[125]	[125], [123, 124][a]	[125]	[125]	[125]	[125], [123, 124][a]	[125], [123, 124][a]	[125], [123, 124][a]	[125], [123, 124][a]

[a]commercial tool.

work is based on reactionary measures and no proactive strategy has been defined. This is one reason why only a few solutions can find zero-day vulnerabilities.

- Not all sub-classes of vulnerabilities are addressed by every solution. For example, there are numerous web-application vulnerabilities and existing solutions if based on static analysis can detect only a subset of web vulnerabilities.
- A wide range of methods are used by different solutions to do the firmware analysis including function profiling, program slicing, relationship between shell scripts, code snippets emulation, and augmented process emulation etc.
- Scalability is an issue in many solutions. The solutions are either time-consuming, takes a lot of resources or requires human intervention to do the analysis. During emulation, there is not

Table 11
Open research issues.

S.#	Issue	Category
1	Unified firmware auditing framework	Standardization
2	Unified firmware stack	Standardization
3	Decompilers for all architectures	Technical
4	Automation of reverse engineering	Technical
5	Improved emulations	Technical
6	Better tools	Technical
7	Metadata collection	Technical
8	Secure firmware update	Technical
9	Emulation support for additional architectures	Design
10	Investigate non-Linux firmware	Design
11	IoT specific operating systems	Design

one good solution than can do automatic emulation of a variant number of firmware images.

- Most solutions are not versatile. i.e. they only cover a particular class or subclass of vulnerabilities. However, there are some solutions like [77,129] that can detect numerous vulnerabilities due to the methodology it adopted. But both of them are based on static analysis and there is always a chance of false positives in static analysis.
- The use of technologies like Artificial Intelligence and machine learning is rare. Therefore, automation and self-improvement will not be possible.

Of course, developing an ideal solution will not be possible. However, achieving to build a framework that will make the evolution towards an ideal solution will definitely help. In the next section, we will discuss the open-issues that researchers can take on to help facilitate the development of a near-ideal solution.

5. Open research issues

This section highlights the obstacles that are slowing down the advent in firmware security. Alternatively, we can look at these obstacles as opportunities for the research and/or open-source development community. Clearing these obstacles will eventually smoothen the road to building a versatile IoT firmware analysis solution. Below, we provide a list of open issues which we have categorized according to the Table 11.

5.1. Unified firmware auditing framework

Numerous solutions are discussed in this article but mostly focus on covering a subset or class of firmware vulnerabilities. Research needs to be carried out to develop frameworks rather solutions that can be a part of the framework. We idealize solutions/frameworks like [30,31,130]. The development of a self-evolving, extendable, platform-independent and automatic framework will facilitate firmware auditing, testing and validation for the IoT vendors. Necessarily, the framework should introduce both static and dynamic analysis and should be platform independent. It would be a good idea to modularize the auditing framework on the basis of classes of IoT devices [131] or platforms. Needless to say, the framework will be able to audit the hardware, firmware and connectivity aspects of the IoT devices for compliance, configurations and implementation issues.

5.2. Unified firmware stack

Firmware (in)security will be rebooted if there is a unified machine-independent stack for firmware development that is followed by IoT device vendors. Even if no new stacks are developed but existing supported stack(s) like Unified Extensible Firmware Interface (UEFI) are followed, it can greatly enhance the security of firmware. For example, literature suggests that there are misconceptions about the usage of Unified Extensible Firmware Interface (UEFI) for IoT devices and it can be trusted by the IoT firmware developers[24] [132]. Nonetheless, it is imperative to design a firmware stack specialized for IoT and its future in mind.

5.3. Automation of reverse engineering

As previously established, reverse engineering is a skill that has to be learned over time. Doing large scale analysis is possible only if some level of automation is reached. One particular field that can support automation is Machine Learning(ML). The additional layer of machine learning during reverse engineering can simplify the task by connecting the dots for humans. Specifically, ML can identify similar problems in different architectures through learning. Furthermore, it can thoroughly test firmware and avoid human err.

5.4. Emulation support for additional architectures

As discussed before, static analysis may generate a lot of false positives and yet skip a number of vulnerabilities that can only be detected during dynamic analysis. It is evident from Section 4 that static analysis based solutions have outnumbered dynamic analysis-based solutions. The primary reason is the inheriting complexity and less support for emulation across multiple architectures in dynamic approach. Research is needed to enable dynamic analysis to support higher number of processor architectures by extending QEMU or similar tools.

5.5. Improved emulations

Emulation is not always flawless. For instance, emulations may crash because of unavailability of NVRAM parameters [38]. Not only is it required to add support for more platforms, it is also indispensable to improve the performance and reliability of existing emulation techniques. As an example, we can quote FEMU, which is a hybrid emulation framework for FGPA(Field-Programmable Gate Array) that can locate and fix bugs during system emulations.

5.6. Decompiling support for all architectures

Certain processor architectures do not have decompiler(s) available for them. For example, Xtensa is one such architecture that does not have an appreciated support for decompilers and debuggers.

5.7. Better tools

Many tools have been discussed in this paper and the papers referenced in our work. But there are a number of issues with these tools. To mention some of the issues we can say that not all the tools perform as expected. Some tools are not meant for embedded or IoT devices exactly [37] whereas other tools are not supported anymore (like Firmware Analysis ToolKit). Furthermore, some tools (like FRAK and Firmalice) are merely names/frameworks and no tool or source code is made available by the authors. Whereas certain great tools like IDA pro are too expensive and not open-source. Additionally, reverse engineering is not a single-tool job but an amalgamation of many different features-rich tools. Apart from all the above issues, even if have there are tools that can be deemed useful, they will have a high false positive/negative rate due to the versatility of IoT devices [85]. Therefore, to unpack, understand, dissect and analyze firmware images for vulnerabilities, effort is needed to further the existing tools and develop other tools to fill the gap.

[24] Firmware security for IoT devices, https://www.embedded-computing.com/articles/firmware-security-for-iot-devices.

5.8. Metadata collection

Not all firmware obtained from the vendors have their associated metadata available with them. Metadata however, is an essential factor for firmware analysis. Researchers and developers need to work on collecting metadata not only for ease in static and dynamic analysis but it can also prove useful for reducing the overall analysis time by leading way towards finding typical vulnerabilities. For instance, a tool can be developed that can provide metadata if a firmware is provided to it as input. The tool may trace back to original vendor website or dig inside the firmware itself to collect information in an automated way. In essence, there is a dire need of a publicly available database of firmware along with their metadata to allow researchers and security experts to benefit from it as well as share their expertise to benefit vendors.

5.9. Secure firmware update

Prominent IoT security organizations like European Union Agency for Cybersecurity (ENISA) [133], OWASP, IoTSF, and Symantec[25] recommend and consider secure IoT firmware update as one of the most important pillar in IoT security. A number of IoT secure update protocols including IETF's SUIT [134] standards have been suggested[26] [135–139]. Although it is a good idea to develop and use a custom secure update mechanism, a standardized firmware update framework for this purpose can seal one of the biggest attack vectors in the IoT paradigm [22]. Research community can fill the gap by introducing an update mechanism that can be used for versatile IoT devices and support a number of architectures yet providing flexibility in updating multiple or all components of IoT firmware.

5.10. Investigate non-Linux firmware

Much of the work done on firmware analysis is for Linux-based systems. Availability of (open source and free) tools for Linux based firmware, understanding and popularity of Linux in general, large market chunk grabbed by Linux-based firmware [140] and the complexity involved in analysis of non-linux based firmware cuts the chances of its analysis. Even after that, a large portion of firmware are still based on non-linux systems. Therefore, for firmware based on other OSes or non-OS based systems, research needs to be carried to invent new methodologies for finding and resolving their vulnerabilities. Essentially, understanding the behavior of the device architecture, the unpacking and format analysis and finally realizing the code behavior can shed some light to resolve the problem. It is still a challenge since the system is developed with no standardization and merely relies on the developer or vendor mindset.

5.11. IoT specific operating systems

The intense hike in the number of IoT devices gave little time to OS designers to come up with an IoT specific operating system design. Current IoT devices mostly use stripped down versions of existing OSes primarily designed for other purposes. Embedded linux, VxWorks are examples of such operating systems. On the other hand, there are several operating systems specifically designed for IoT (For example, Contiki, Android things and LiteOS etc.). Since the hardware resources spectrum in IoT device is large due to its inheriting versatility, different (secure) IoT operating systems should be designed and developed accordingly. Essentially, it cannot be possible to always use the same operating system within the IoT constrained device as well as the IoT gateway device. Furthermore, there should be a harmony between the operating system and firmware design and standards.

6. Recommendations

Working on the research issues can certainly help the cause of security of IoT firmware in the long run. However, this section provides some suggestions and straight-forward measures that can be used by the manufacturers along with other stakeholders to develop a secure and robust firmware. Following is the list of recommendations that we think can make an impact:

- The choice of OS is a crucial decision when creating an IoT device. Therefore, it should be based on some solid grounds rather than done randomly. According to Afzal, the choice of OS should be made based on underlying hardware architecture [141]. Developers can make the optimal selection of OS after performing a feature-mapping for each OS. Typical features include inter-process communication, power efficiency, robustness against bugs, security and privacy, etc. The right choice of OS will enable full exploitation of the underlying hardware and make various SDKs and drivers readily available for development and support. Furthermore, the choice of OS is not the only precaution that can help. Removing all the unnecessary libraries and packages that are not going to be needed will further reduce the possibility of potential exploits.
- In order to get all the help from the research community and adapt to the new trend of collaborative ecosystem effort, it will be essentially a positive step to make the firmware of IoT devices public[27] [51]. Providing a source code and design of the firmware will encourage researchers and developers to contribute as well as pave the way to standardization of firmware stacks.
- It is possible to use Blockchain technology for detecting tampering of IoT firmware. Since firmware can be remotely modified by an attacker, storing metadata of firmware on a blockchain along with its checksum can be used to detect firmware tampering [142]. Although, the technology of blockchain has been considered for firmware updates [143], it is yet to be experimented for other purposes.
- OEMs and ODMs can develop secure IoT devices by adhering to the usage of standard security protocols and updated software codebase. For example, as stated before, UEFI can enhance security without affecting the memory requirements and performance. Not only will its usage have a positive impact on security of IoT firmware, but it will also enable IoT firmware developers not to start from scratch since the usage of UEFI is well-known in traditional computing devices. Also, employing Cross-Domain Access Control (CDAC) protocols like Shibboleth [144], xDAuth [145], OAuth [146] will substantially improve the security of IoT devices [147].
- The update of firmware should be done using secure OTA update mechanisms. On the other hand, useless hardware interfaces that can allow code dumping should be avoided.
- Proper testing of code should be done using tools carved specially for vulnerability finding purposes during its development and before its release. For example, Flawfinder tool can be used to identify potential security issues in a C/C++ code. Furthermore, manufacturers can employ static and dynamic analysis in a much better fashion during the development lifecycle of an IoT firmware as depicted in Fig. 8. Specifically, static analysis should be used during the development phase [72] while dynamic analysis technique after the product is developed for validation and testing purposes. During a development phase, as depicted in the figure, the usage of static/dynamic analysis increase as we move towards the completion of that phase.

[25] ISTR ransomware & business 2016, http://www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/ISTR2016_Ransomware_and_Businesses.pdf.

[26] Github: Uptane, https://github.com/uptane.

[27] Helping researchers with IoT firmware vulnerability discovery, https://www.helpnetsecurity.com/2018/11/19/iot-firmware-vulnerability-discovery/.

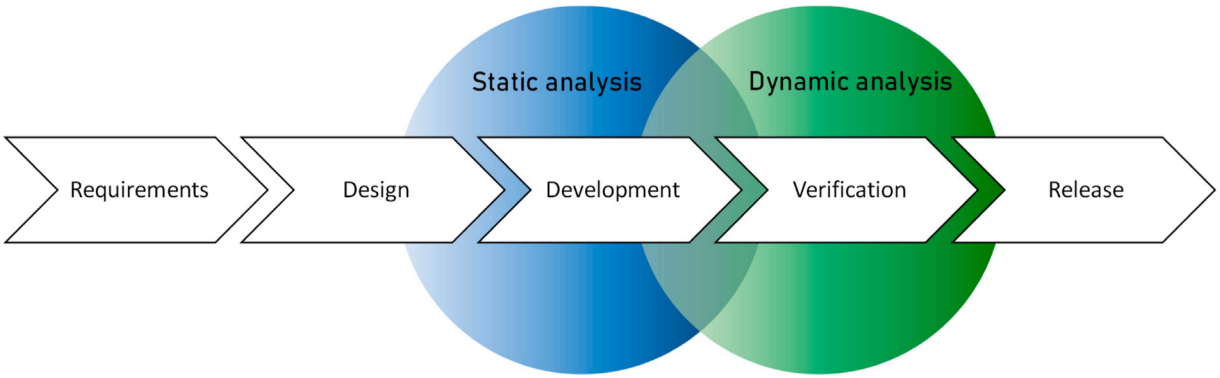


Fig. 8. Ideal usage of static and dynamic analysis in the development lifecycle of IoT firmware.

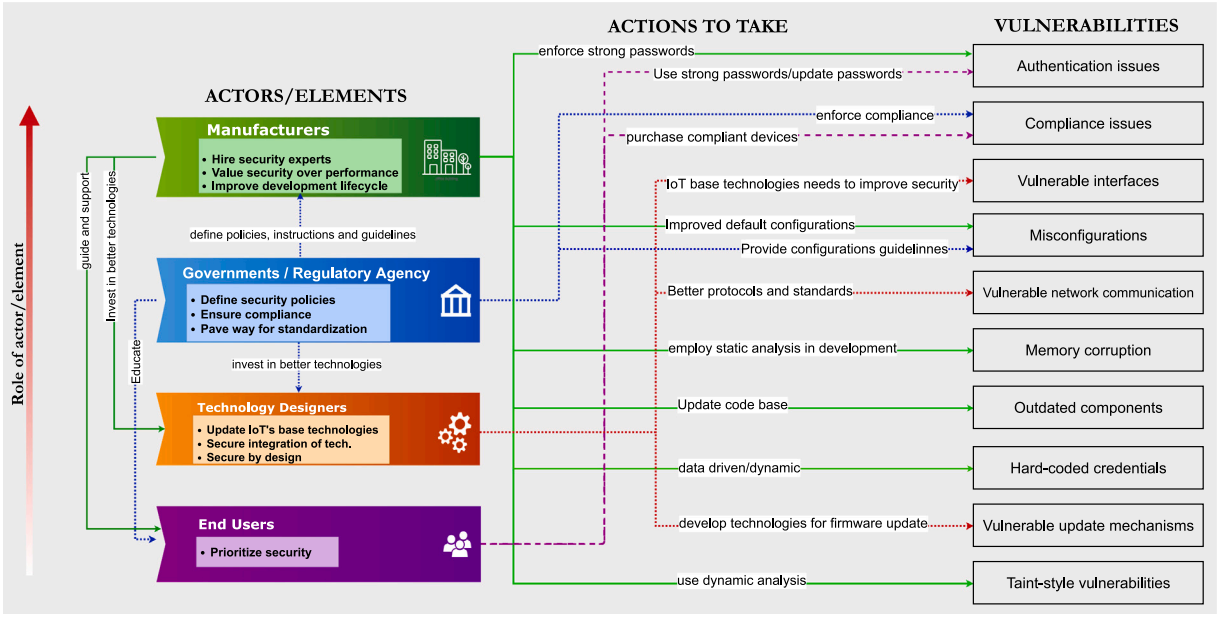


Fig. 9. Major IoT actors and their associated action items to resolve various vulnerabilities.

Apart from these directions, it is important to realize the importance of role of different actors/elements in the development of an IoT system. We provide Fig. 9 to depict how some of the key *actors* can take significant *actions* to fix the well-known vulnerabilities we highlighted earlier. We believe that technology designers, manufacturers, and regulatory authorities have a huge responsibility to improve the overall ecosystem of IoT devices. It is also important to know that other stakeholders like investors can further improve security by allowing companies more time to develop products with enhanced security level.

7. Conclusion

Prevalent use of IoT devices to ease life and achieve automation is foreseeable. However, the safe daily operation of these devices requires an adequate level of security. Improving IoT firmware security can provide bona fide insurance to IoT devices against typical attacks. During the course of this article, we have presented a number of concepts that emphasize the importance of firmware security. Notably, we have explored some critical points in firmware design and development that lead to insecurity in IoT devices.

Furthermore, this work has presented the challenges of doing a thorough firmware security analysis. We have also assessed the strengths and weaknesses of existing solutions and concluded that there is a dire need for a versatile IoT firmware auditing framework. Although we recommended some crucial guidelines for IoT vendors, developers, integrators, and other stakeholders, we believe standardization can resolve significant security loopholes in IoT firmware.

We have established solid grounds to develop a framework for auditing IoT firmware against different classes of security vulnerabilities through this work. Our future endeavors will include the development of a framework that audits not only the application of an IoT device but also the hardware and network connectivity protocols.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research work is supported by grant from the National Center for Cyber Security (NCCS) of Pakistan.

References

[1] O. Arias, J. Wurm, K. Hoang, Y. Jin, Privacy and security in internet of things and wearable devices, IEEE Trans. Multi-Scale Comput. Syst. 1 (2) (2015) 99–109.

[2] M. Hung, Leading the iot, gartner insights on how to lead in a connected world, Gartner Res. (2017) 1–29.
[3] CISCO, Internet of things at a glance, 2016.
[4] I. Lee, K. Lee, The Internet of Things (IoT): Applications, investments, and challenges for enterprises, Bus. Horiz. 58 (4) (2015) 431–440.
[5] N. Kshetri, The evolution of the internet of things industry and market in China: An interplay of institutions, demands and supply, Telecommun. Policy 41 (1) (2017) 49–67.
[6] Y. Shoshitaishvili, R. Wang, C. Hauser, C. Kruegel, G. Vigna, Firmalice-automatic detection of authentication bypass vulnerabilities in binary firmware, in: NDSS, 2015.
[7] F. Zafar, H.A. Khattak, M. Aloqaily, R. Hussain, Carpooling in connected and autonomous vehicles: Current solutions and future directions, ACM Comput. Surv. (2021).
[8] L. Atzori, A. Iera, G. Morabito, The internet of things: A survey, Comput. Netw. 54 (15) (2010) 2787–2805.
[9] K. Pretz, The next evolution of the internet, IEEE Mag. Inst. 50 (5) (2013).
[10] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, et al., Sok:(state of) the art of war: Offensive techniques in binary analysis, in: 2016 IEEE Symposium on Security and Privacy (SP), IEEE, 2016, pp. 138–157.
[11] D. Miessler, Securing the internet of things: Mapping attack surface areas using the OWASP IoT top 10, in: RSA Conference, 2015.
[12] A. Gupta, The IoT Hacker's Handbook, Springer, 2019.
[13] I. Makhdoom, M. Abolhasan, J. Lipman, R.P. Liu, W. Ni, Anatomy of threats to the internet of things, IEEE Commun. Surv. Tutor. 21 (2) (2018) 1636–1675.
[14] H.A. Abdul-Ghani, D. Konstantas, M. Mahyoub, A comprehensive IoT attacks survey based on a building-blocked reference model, IJACSA Int. J. Adv. Comput. Sci. Appl. 9 (3) (2018) 355–373.
[15] M. Ammar, G. Russello, B. Crispo, Internet of Things: A survey on the security of IoT frameworks, J. Inf. Secur. Appl. 38 (2018) 8–27.
[16] V. Adat, B.B. Gupta, Security in Internet of Things: issues, challenges, taxonomy, and architecture, Telecommun. Syst. 67 (3) (2018) 423–441, http://dx.doi.org/10.1007/s11235-017-0345-9.
[17] Z. Ling, K. Liu, Y. Xu, C. Gao, Y. Jin, C. Zou, X. Fu, W. Zhao, IoT security: an end-to-end view and case study, 2018, arXiv preprint arXiv:1805.05853.
[18] J. Gubbi, R. Buyya, S. Marusic, M. Palaniswami, Internet of Things (IoT): A vision, architectural elements, and future directions, Future Gener. Comput. Syst. 29 (7) (2013) 1645–1660.
[19] C. Kolias, G. Kambourakis, A. Stavrou, J. Voas, DDoS in the IoT: Mirai and other botnets, Computer 50 (7) (2017) 80–84.
[20] A. Mohanty, I. Obaidat, F. Yilmaz, M. Sridhar, Control-hijacking vulnerabilities in IoT firmware: A brief survey, in: Proceedings of the 1st International Workshop on Security and Privacy for the Internet-of-Things, IoTSec, 2018.
[21] S. Siboni, V. Sachidananda, Y. Meidan, M. Bohadana, Y. Mathov, S. Bhairav, A. Shabtai, Y. Elovici, Security testbed for internet-of-things devices, IEEE Trans. Reliab. 68 (1) (2019) 23–44.
[22] K. Zandberg, K. Schleiser, F. Acosta, H. Tschofenig, E. Baccelli, Secure firmware updates for constrained iot devices using open standards: A reality check, IEEE Access 7 (2019) 71907–71920.
[23] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, N. Ghani, Demystifying IoT security: an exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations, IEEE Commun. Surv. Tutor. 21 (3) (2019) 2702–2733.
[24] M. Yu, J. Zhuge, M. Cao, Z. Shi, L. Jiang, A survey of security vulnerability analysis, discovery, detection, and mitigation on IoT devices, Future Internet 12 (2) (2020) 27.
[25] A. Costin, Security of cctv and video surveillance systems: Threats, vulnerabilities, attacks, and mitigations, in: Proceedings of the 6th International Workshop on Trustworthy Embedded Devices, 2016, pp. 45–54.
[26] A. Costin, J. Zaddach, Iot malware: Comprehensive survey, analysis framework and case studies, BlackHat USA (2018).
[27] W. Xie, Y. Jiang, Y. Tang, N. Ding, Y. Gao, Vulnerability detection in IoT firmware: A survey, in: Parallel and Distributed Systems (ICPADS), 2017 IEEE 23rd International Conference on, IEEE, 2017, pp. 769–772.
[28] M. Muench, J. Stijohann, F. Kargl, A. Francillon, D. Balzarotti, What you corrupt is not what you crash: Challenges in fuzzing embedded devices, in: NDSS, 2018.
[29] C. Wright, W.A. Moeglein, S. Bagchi, M. Kulkarni, A.A. Clements, Challenges in firmware re-hosting, emulation, and analysis, ACM Comput. Surv. 54 (1) (2021) https://doi.org/10.1145/3423167.
[30] I. Nadir, Z. Ahmad, H. Mahmood, G.A. Shah, F. Shahzad, M. Umair, H. Khan, U. Gulzar, An auditing framework for vulnerability analysis of IoT system, in: 2019 IEEE European Symposium on Security and Privacy Workshops (EuroS PW), 2019, pp. 39–47, https://doi.org/10.1109/EuroSPW.2019.00011.
[31] Z. Wang, Y. Zhang, Z. Tian, Q. Ruan, T. Liu, H. Wang, Z. Liu, J. Lin, B. Fang, W. Shi, Automated vulnerability discovery and exploitation in the internet of things, Sensors 19 (15) (2019) 3362.
[32] A. Costin, J. Zaddach, A. Francillon, D. Balzarotti, S. Antipolis, A large-scale analysis of the security of embedded firmwares, in: USENIX Security Symposium, 2014, pp. 95–110.
[33] A. Costin, Large scale security analysis of embedded devices' firmware, ECOM ParisTech (2015).
[34] L. Cojocar, J. Zaddach, R. Verdult, H. Bos, A. Francillon, D. Balzarotti, PIE: PArser identification in embedded systems, in: Proceedings of the 31st Annual Computer Security Applications Conference, 2015, pp. 251–260.
[35] S.L. Thomas, F.D. Garcia, T. Chothia, HumIDIFy: a tool for hidden functionality detection in firmware, in: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Springer, 2017, pp. 279–300.
[36] Y. David, N. Partush, E. Yahav, Firmup: Precise static detection of common vulnerabilities in firmware, in: ACM SIGPLAN Notices, vol. 53, ACM, 2018, pp. 392–404.
[37] A. Costin, A. Zarras, A. Francillon, Automated dynamic firmware analysis at scale: a case study on embedded web interfaces, in: Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security, ACM, 2016, pp. 437–448.
[38] J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W.C. Lau, M. Sun, R. Yang, K. Zhang, IoTFuzzer: Discovering memory corruptions in IoT through app-based fuzzing, in: NDSS, 2018.
[39] J. Zaddach, L. Bruno, A. Francillon, D. Balzarotti, et al., AVATAR: A Framework to support dynamic security analysis of embedded systems' firmwares, in: NDSS, vol. 14, 2014, pp. 1–16.
[40] L. Zhu, X. Fu, Y. Yao, Y. Zhang, H. Wang, FIoT: Detecting the memory corruption in lightweight IoT device firmware, in: 2019 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/13th IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE), IEEE, 2019, pp. 248–255.
[41] Z. Gui, H. Shu, F. Kang, X. Xiong, FIRMCORN: VUlnerability-oriented fuzzing of IoT firmware via optimized virtual execution, IEEE Access (2020).
[42] O. Shwartz, Y. Mathov, M. Bohadana, Y. Elovici, Y. Oren, Reverse engineering IoT devices: Effective techniques and methods, IEEE Internet Things J. 5 (6) (2018) 4965–4976.
[43] J. Zaddach, A. Costin, Embedded devices security and firmware reverse engineering, Black-Hat USA (2013).
[44] G. Palavicini Jr., J. Bryan, E. Sheets, M. Kline, J. San Miguel, Towards firmware analysis of industrial internet of things (IIoT), 2017, Special Session on Innovative CyberSecurity and Privacy for Internet of Things: Strategies, Technologies, and Implementations.
[45] S. Vasile, D. Oswald, T. Chothia, Breaking all the things—A systematic survey of firmware extraction techniques for IoT devices, in: International Conference on Smart Card Research and Advanced Applications, Springer, 2018, pp. 171–185.
[46] A. Milinković, S. Milinković, L. Lazić, Choosing the right RTOS for IoT platform, Infoteh-Jahorina 14 (2015) 504–509.
[47] A. Guzman, A. Gupta, IoT Penetration Testing Cookbook: Identify Vulnerabilities and Secure Your Smart Devices, Packt Publishing Ltd, 2017.
[48] A. Manske, Conducting a vulnerability assessment of an IP camera, 2019.
[49] D.D. Chen, M. Woo, D. Brumley, M. Egele, Towards automated dynamic analysis for linux-based embedded firmware, in: NDSS, 2016, pp. 1–16.
[50] J. Zaddach, A. Costin, Embedded devices security and firmware reverse engineering, Black-Hat USA (2013).
[51] V. Zimmer, J. Sun, M. Jones, S. Reinauer, Embedded Firmware Solutions: Development Best Practices for the Internet of Things, A Press, 2015.
[52] P. Suresh, J.V. Daniel, V. Parthasarathy, R. Aswathy, A state of the art review on the internet of things (IoT) history, technology and fields of deployment, in: 2014 International Conference on Science Engineering and Management Research, ICSEMR, IEEE, 2014, pp. 1–8.
[53] G.H. Sockut, Firmware/hardware support for operating systems: principles and selected history, ACM SIGMICRO Newsl. 6 (4) (1975) 17–26.
[54] A. Chatzigeorgiou, G. Stephanides, Evaluating performance and power of object-oriented vs. procedural programming in embedded processors, in: International Conference on Reliable Software Technologies, Springer, 2002, pp. 65–75.
[55] A. Gilchrist, IoT Security Issues, Walter de Gruyter GmbH & Co KG, 2017.
[56] L. Cojocar, J. Zaddach, R. Verdult, H. Bos, A. Francillon, D. Balzarotti, Pie: parser identification in embedded systems, in: Proceedings of the 31st Annual Computer Security Applications Conference, ACM, 2015, pp. 251–260.
[57] N.G. Tsoutsos, M. Maniatakos, Anatomy of memory corruption attacks and mitigations in embedded systems, IEEE Embed. Syst. Lett. 10 (3) (2018) 95–98.
[58] J. Zaddach, L. Bruno, A. Francillon, D. Balzarotti, et al., AVATAR: A framework to support dynamic security analysis of embedded systems' firmwares, in: NDSS, 2014.
[59] S.L. Thomas, T. Chothia, F.D. Garcia, Stringer: measuring the importance of static data comparisons to detect backdoors and undocumented functionality, in: European Symposium on Research in Computer Security, Springer, 2017, pp. 513–531.
[60] A. Greenberg, The reaper iot botnet has already infected a million networks, 2017, URL: https://www.wired.com/story/reaper-iot-botnet-infected-million-networks/ (visited on 01/13/2018).
[61] M.B. Barcena, C. Wueest, Insecurity in the Internet of Things, Secur. Response Symantec (2015) 20.
[62] M. Kol, JSOF research lab, 2020.
[63] M. Lezzi, M. Lazoi, A. Corallo, Cybersecurity for Industry 4.0 in the current literature: A reference framework, Comput. Ind. 103 (2018) 97–110.

[64] J. Classen, D. Wegemer, P. Patras, T. Spink, M. Hollick, Anatomy of a vulnerable fitness tracking system: Dissecting the fitbit cloud, app, and firmware, Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. 2 (1) (2018) 1–24.

[65] E. Bertino, N. Islam, Botnets and internet of things security, Computer 50 (2) (2017) 76–79.

[66] A. Mandal, P. Ferrara, Y. Khlyebnikov, A. Cortesi, F. Spoto, Cross-program taint analysis for IoT systems, in: Proceedings of the 35th Annual ACM Symposium on Applied Computing, 2020, pp. 1944–1952.

[67] I. Sutherland, G.E. Kalb, A. Blyth, G. Mulley, An empirical examination of the reverse engineering process for binary files, Comput. Secur. 25 (3) (2006) 221–228.

[68] M. Egele, T. Scholte, E. Kirda, C. Kruegel, A survey on automated dynamic malware-analysis techniques and tools, ACM Comput. Surv. 44 (2) (2012) 6.

[69] M. Popa, Binary code disassembly for reverse engineering, J. Mob. Embed. Distrib. Syst. 4 (4) (2012) 233–248.

[70] M. Serrano, Lecture notes on decompilation, 2013.

[71] A. Petukhov, D. Kozlov, Detecting Security Vulnerabilities in Web Applications Using Dynamic Analysis with Penetration Testing, Computing Systems Lab, Department of Computer Science, Moscow State University, 2008, pp. 1–120.

[72] B. Chess, G. McGraw, Static analysis for security, IEEE Secur. Priv. 2 (6) (2004) 76–79.

[73] S.C. Johnson, Lint, A C Program Checker, Citeseer, 1977.

[74] H. Chen, D. Dean, D.A. Wagner, Model checking one million lines of C code, in: NDSS, vol. 4, 2004, pp. 171–185.

[75] M.C. Grace, Y. Zhou, Z. Wang, X. Jiang, Systematic detection of capability leaks in stock android smartphones, in: NDSS, vol. 14, 2012, p. 19.

[76] C. May, E. Silha, R. Simpson, H. Warren, et al., The PowerPC Architecture: A Specification for a New Family of RISC Processors, Morgan Kaufmann Publishers Inc., 1994.

[77] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, H. Yin, Scalable graph-based bug search for firmware images, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 480–491.

[78] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, D. Song, Neural network-based graph embedding for cross-platform binary code similarity detection, 2017, CoRR arXiv:1708.06525.

[79] Y. Wang, J. Shen, J. Lin, R. Lou, Staged method of code similarity analysis for firmware vulnerability detection, IEEE Access 7 (2019) 14171–14185.

[80] C.-W. Tien, T.-T. Tsai, Y. Chen, S.-Y. Kuo, Ufo-hidden backdoor discovery and security verification in IoT device firmware, in: 2018 IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW, IEEE, 2018, pp. 18–23.

[81] C. Cadar, D. Dunbar, D.R. Engler, et al., KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, in: OSDI, vol. 8, 2008, pp. 209–224.

[82] F. Bellard, QEMU, a fast and portable dynamic translator, in: USENIX Annual Technical Conference, FREENIX Track, vol. 41, 2005, p. 46.

[83] D. Davidson, B. Moench, T. Ristenpart, S. Jha, {FIE} on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution, 2013, pp. 463–478, URL https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/davidson.

[84] H. Moore, Metasploitable 2 exploitability guide, 2012, p. 2013, Retrieved June 27.

[85] J.-b. Hou, T. Li, C. Chang, Research for vulnerability detection of embedded system firmware, Procedia Comput. Sci. 107 (2017) 814–818.

[86] M. Wolfgang, Host discovery with nmap, in: Exploring Nmap's Default Behavior, vol. 1, 2002, p. 16.

[87] R. Rogers, Nessus Network Auditing, Elsevier, 2011.

[88] G. Palavicini Jr., J. Bryan, E. Sheets, M. Kline, J. San Miguel, Towards firmware analysis of industrial internet of things (IIoT), 2017, Special Session on Innovative CyberSecurity and Privacy for Internet of Things: Strategies, Technologies, and Implementations.

[89] Y. Shoshitaishvili, A. Bianchi, K. Borgolte, A. Cama, J. Corbetta, F. Disperati, A. Dutcher, J. Grosen, P. Grosen, A. Machiry, et al., Mechanical phish: Resilient autonomous hacking, IEEE Secur. Priv. 16 (2) (2018) 12–22.

[90] M. Zalewski, American fuzzy lop, 2017, URL: http://lcamtuf.coredump.cx/afl.

[91] T.R. Alves, M. Buratto, F.M. de Souza, T.V. Rodrigues, Openplc: An open source alternative to automation, in: IEEE Global Humanitarian Technology Conference, GHTC 2014, IEEE, 2014, pp. 585–589.

[92] M. Zheng, M. Sun, J. Lui, DroidRay: a security evaluation system for customized android firmwares, in: Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security, ACM, 2014, pp. 471–482.

[93] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, L. Sun, FIRM-AFL: High-throughput greybox fuzzing of iot firmware via augmented process emulation, in: 28th {USENIX} Security Symposium ({USENIX} Security 19), 2019, pp. 1099–1114.

[94] D. Maier, B. Radtke, B. Harren, Unicorefuzz: On the viability of emulation for kernelspace fuzzing, in: 13th USENIX Workshop on Offensive Technologies (WOOT 19), USENIX Association, Santa Clara, CA, 2019, URL https://www.usenix.org/conference/woot19/presentation/maier.

[95] J.-C. Fonbonne, Automated and remote security fuzz testing tool for IoT devices, in: C&ESAR 2016, 2016, p. 133.

[96] D.-i. Jang, T. Kim, H. Kim, A design of IoT protocol fuzzer, in: Advanced Multimedia and Ubiquitous Engineering, Springer, 2017, pp. 242–246.

[97] N. Karamchandani, Mutation based protocol fuzzer for IoT, 2017.

[98] B. Yu, P. Wang, T. Yue, Y. Tang, Poster: Fuzzing iot firmware via multi-stage message generation, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2525–2527.

[99] P. Srivastava, H. Peng, J. Li, H. Okhravi, H. Shrobe, M. Payer, Firmfuzz: Automated iot firmware introspection and analysis, in: Proceedings of the 2nd International ACM Workshop on Security and Privacy for the Internet-of-Things, 2019, pp. 15–21.

[100] Y. Zheng, Z. Song, Y. Sun, K. Cheng, H. Zhu, L. Sun, An efficient greybox fuzzing scheme for linux-based iot programs through binary static analysis, in: 2019 IEEE 38th International Performance Computing and Communications Conference, IPCCC, IEEE, 2019, pp. 1–8.

[101] D. Wang, X. Zhang, T. Chen, J. Li, Discovering vulnerabilities in COTS IoT devices through blackbox fuzzing web management interface, Secur. Commun. Netw. 2019 (2019).

[102] Y. Yao, W. Zhou, Y. Jia, L. Zhu, P. Liu, Y. Zhang, Identifying privilege separation vulnerabilities in IoT firmware with symbolic execution, in: European Symposium on Research in Computer Security, Springer, 2019, pp. 638–657.

[103] Z. Gui, H. Shu, J. Yang, Firmnano: Toward iot firmware fuzzing through augmented virtual execution, in: 2020 IEEE 11th International Conference on Software Engineering and Service Science, ICSESS, IEEE, 2020, pp. 290–294.

[104] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, Y. Kim, Firmae: Towards large-scale emulation of iot firmware for dynamic analysis, in: Annual Computer Security Applications Conference, 2020, pp. 733–745.

[105] Z. Gao, W. Dong, R. Chang, Y. Wang, Fw-fuzz: A code coverage-guided fuzzing framework for network protocols on firmware, Concurr. Comput.: Pract. Exper. (2020).

[106] B. Feng, A. Mera, L. Lu, {P2IM}: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling, in: 29th USENIX Security Symposium (USENIX Security 20), 2020, pp. 1237–1254.

[107] H.-W. Kim, J.-H. Kim, J. Yun, Efficient coverage guided IoT firmware fuzzing technique using combined emulation, J. Korea Inst. Inf. Secur. Cryptol. 30 (5) (2020) 847–857.

[108] M. Börsig, S. Nitzsche, M. Eisele, R. Gröll, J. Becker, I. Baumgart, Fuzzing framework for ESP32 microcontrollers, in: 2020 IEEE International Workshop on Information Forensics and Security, WIFS, IEEE, 2020, pp. 1–6.

[109] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of iot firmware via message snippet inference, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 337–350.

[110] H. Zhang, K. Lu, X. Zhou, Q. Yin, P. Wang, T. Yue, Siotfuzzer: Fuzzing web interface in iot firmware via stateful message generation, Appl. Sci. 11 (7) (2021) 3120.

[111] J. Kim, J. Yu, H. Kim, F. Rustamov, J. Yun, FIRM-COV: High-coverage greybox fuzzing for IoT firmware via optimized process emulation, IEEE Access 9 (2021) 101627–101642.

[112] P. Liu, S. Ji, X. Zhang, Q. Dai, K. Lu, L. Fu, W. Chen, P. Cheng, W. Wang, R. Beyah, Ifizz: Deep-state and efficient fault-scenario generation to test IoT firmware, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 805–816.

[113] Q. Yin, X. Zhou, H. Zhang, Firmhunter: State-aware and introspection-driven grey-box fuzzing towards IoT firmware, Appl. Sci. 11 (19) (2021) 9094.

[114] R. Fan, J. Pan, S. Huang, Arm-afl: coverage-guided fuzzing framework for ARM-based IoT devices, in: International Conference on Applied Cryptography and Network Security, Springer, 2020, pp. 239–254.

[115] E. Hwang, H. Lee, S. Jeong, M. Cho, T. Kwon, Toward fast and scalable firmware fuzzing with dual-level peripheral modeling, IEEE Access 9 (2021) 141790–141799.

[116] A. Mera, B. Feng, L. Lu, E. Kirda, Dice: Automatic emulation of dma input channels for dynamic firmware analysis, in: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, 2021, pp. 1938–1954.

[117] Q. Yin, X. Zhou, H. Zhang, An efficient feedback-enhanced fuzzing scheme for linux-based IoT firmwares, in: 2021 IEEE 21st International Conference on Communication Technology, ICCT, IEEE, 2021, pp. 754–758.

[118] M. Kil, Log-based light-weight fuzzer for IoT firmware, 2021.

[119] K. Cheng, Q. Li, L. Wang, Q. Chen, Y. Zheng, L. Sun, Z. Liang, Dtaint: detecting the taint-style vulnerability in embedded device firmware, in: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, IEEE, 2018, pp. 430–441.

[120] Z.B. Celik, L. Babun, A.K. Sikder, H. Aksu, G. Tan, P. McDaniel, A.S. Uluagac, Sensitive information tracking in commodity IoT, in: 27th {USENIX} Security Symposium ({USENIX} Security 18), 2018, pp. 1687–1704.

[121] GitHub - craigz28/firmwalker: Script for searching the extracted firmware file system for goodies!, 2020, https://github.com/craigz28/firmwalker. (Accessed 28 July 2020).

[122] GitHub - CERTCC/trommel: TROMMEL: SIft through embedded device files to identify potential vulnerable indicators, 2020, https://github.com/CERTCC/trommel. (Accessed on 28 July 2020).
[123] Security analysis for IoT devices — completely automated — IoT inspector, 2020, https://www.iot-inspector.com/. (Accessed 4 August 2020).
[124] Firmalyzer enterprise PLATFORM: The first automated firmware security analysis platform for connected devices, 2020, https://firmalyzer.com/product/. (Accessed 4 August 2020).
[125] D. Yu, L. Zhang, Y. Chen, Y. Ma, J. Chen, Large-scale IoT devices firmware identification based on weak password, IEEE Access 8 (2020) 7981–7992.
[126] C. Zhang, Y. Li, H. Chen, X. Luo, M. Li, A.Q. Nguyen, Y. Liu, BIFF: PRactical binary fuzzing framework for programs of IoT and mobile devices, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 1161–1165.
[127] C. Păduraru, R. Cristea, E. Stăniloiu, RiverIoT-a framework proposal for fuzzing IoT applications, in: 2021 IEEE/ACM 3rd International Workshop on Software Engineering Research and Practices for the IoT, SERP4IoT, IEEE, 2021, pp. 52–58.
[128] X. Liu, B. Cui, J. Fu, J. Ma, HFuzz: Towards automatic fuzzing testing of NB-IoT core network protocols implementations, Future Gener. Comput. Syst. 108 (2020) 390–400.
[129] H. Wang, Z. Zhang, T. Taleb, Special issue on security and privacy of IoT, World Wide Web 21 (1) (2018) 1–6.
[130] G. Lally, D. Sgandurra, Towards a framework for testing the security of IoT devices consistently, in: International Workshop on Emerging Technologies for Authorization and Authentication, Springer, 2018, pp. 88–102.
[131] C. Bormann, M. Ersue, A. Keranen, RFC 7228: Terminology for constrained-node networks, IETF Request for Comments (2014).
[132] M. Krau, D. Wei, Clarifying the Ten Most Common Misconceptions About UEFI, April, 2014.
[133] E. ENISA, Baseline Security Recommendations for IoT in the Context of Critical Information Infrastructures, 2017.
[134] B. Moran, M. Meriac, H. Tschofenig, D. Brown, A firmware update architecture for Internet of Things devices, in: Internet-Draft draft-moran-suit-architecture-02, IETF, 2019.
[135] L. Morel, D. Couroussé, Idols with Feet of Clay: On the Security of Bootloaders and Firmware Updaters for the IoT.
[136] H. Chandra, E. Anggadjaja, P.S. Wijaya, E. Gunawan, Internet of Things: Over-the-Air (OTA) firmware update in Lightweight mesh network protocol for smart urban development, in: 2016 22nd Asia-Pacific Conference on Communications, APCC, IEEE, 2016, pp. 115–118.
[137] X. He, S. Alqahtani, R. Gamble, M. Papa, Securing over-the-air IoT firmware updates using blockchain, in: Proceedings of the International Conference on Omni-Layer Intelligent Systems, 2019, pp. 164–171.
[138] G. Jurkovic, V. Sruk, Remote firmware update for constrained embedded systems, in: 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO, IEEE, 2014, pp. 1019–1023.
[139] S. Schmidt, M. Tausig, M. Hudler, G. Simhandl, Secure firmware update over the air in the internet of things focusing on flexibility and feasibility, in: Internet of Things Software Update Workshop (IoTSU). Proceeding, 2016.
[140] I. Skerrett, IoT Developer Survey 2016, Eclipse IoT Working Group, IEEE IoT and Agile IoT, 2016, pp. 1–39.
[141] B. Afzal, M. Umair, G.A. Shah, E. Ahmed, Enabling IoT platforms for social IoT applications: vision, feature mapping, and challenges, Future Gener. Comput. Syst. 92 (2019) 718–731.
[142] J.-M. Lim, Y. Kim, C. Yoo, Chain veri: Blockchain-based firmware verification system for IoT environment, in: 2018 IEEE International Conference on Internet of Things (IThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), 2018, pp. 1050–1056, http://dx.doi.org/10.1109/Cybermatics_2018.2018.00194.
[143] B. Lee, J.-H. Lee, Blockchain-based secure firmware update for embedded devices in an internet of things environment, J. Supercomput. 73 (3) (2017) 1152–1167.
[144] J. Jensen, M.G. Jaatun, Federated identity management-we built it; why won't they come? IEEE Secur. Priv. 11 (2) (2012) 34–41.
[145] M. Alam, X. Zhang, K. Khan, G. Ali, Xdauth: a scalable and lightweight framework for cross domain access control and delegation, in: Proceedings of the 16th ACM Symposium on Access Control Models and Technologies, 2011, pp. 31–40.
[146] B. Leiba, "Oauth web authorization protocol". IEEE internet computing. Nicosia, cyprus, 2012.
[147] S. Zahra, W. Gong, H.A. Khattak, M.A. Shah, H. Song, Cross-domain security and interoperability in internet of things, IEEE Internet Things J. (2021).